Learning Declarative Rules for Collective Entity Resolution: A Learning from Failures Approach

Zhiliang Xiang
Cardiff University
Cardiff, United Kingdom
xiangz6@cardiff.ac.uk

Meghyn Bienvenu
Univ. Bordeaux, CNRS, Bordeaux INP,
LaBRI, UMR 5800, Talence, France
meghyn.bienvenu@u-bordeaux.fr

Víctor Gutiérrez-Basulto
Cardiff University
Cardiff, United Kingdom
gutierrezbasultov@cardiff.ac.uk

Yazmín Ibáñez-García
Cardiff University
Cardiff, United Kingdom
ibanezgarciay@cardiff.ac.uk

Katsumi Inoue
National Institute of Informatics
Tokyo, Japan
inoue@nii.ac.jp

ABSTRACT

This paper studies learning declarative rules for collective entity resolution (ER). Unlike pairwise ER rules that perform a single pass of pairwise comparisons within a single table, collective ER rules jointly resolve entity references across multiple relations in a recursive manner. To learn such rules, we propose a Learning from Failures-based framework in which candidate ER rules are generated using Answer Set Programming. The search space is then pruned by deriving declarative constraints from candidate rules that fail to satisfy predefined success criteria, thereby eliminating their generalisations or specialisations according to a subsumption ordering. This yields an efficient learning algorithm to compute an optimal ruleset that best generalises the labeled examples while minimising its size. Experimental results show that our approach learns concise, effective multi-relational rules. Crucially, by leveraging denial constraints, our framework compensates for underfitting when trained on a mere 20% of the data, recovering performance close to that achieved using the full training set.

PVLDB Reference Format:
Zhiliang Xiang, Meghyn Bienvenu, Víctor Gutiérrez-Basulto, Yazmín Ibáñez-García, and Katsumi Inoue. Learning Declarative Rules for Collective Entity Resolution: A Learning from Failures Approach. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/zl-xiang/ER-rule-mining.

1 INTRODUCTION

Entity resolution (ER) is a fundamental data quality problem focused on identifying distinct database constants that refer to the same real-world entity [6, 14, 31, 37]. While traditional methods focus on pairwise matching within a single relation, *collective* ER jointly resolves entity references across multiple relations [8, 9, 19]. This collective approach is crucial in practice, as many duplicates can only be identified by recursively exploiting dependencies among previously resolved entities [19, 57].

To address this multi-faceted problem, diverse techniques have been proposed [11, 14], spanning probabilistic models [28, 54], machine learning [8, 39, 46, 49, 59], and declarative frameworks using rules or integrity constraints [3, 9, 19, 57, 58]. Recently, pre-trained large language models (LLMs) have significantly advanced pairwise matching, often yielding strong results zero-shot [41, 46, 49]. However, these LLM-based approaches are largely restricted to pairwise settings and rely on opaque black boxes, limiting their reliability and explainability [56]. This lack of transparency motivates a growing demand for responsible, explainable ER paradigms capable of providing justifiable decisions, especially in high-stakes domains like healthcare and law [13].

Declarative approaches are well-suited to meet these demands in complex, multi-relational settings. They naturally exploit underlying relational dependencies [9, 19, 57] and offer clear explainability, which helps handle data noise [11, 57]. Yet, their practical adoption remains severely bottlenecked by the labor-intensive knowledge engineering required to manually craft high-quality rules. Surprisingly, while automatic rule synthesis has been researched to alleviate this bottleneck for pairwise ER [38, 53], the automatic discovery of collective ER rules has received little attention.

Challenges. Learning collective ER rules introduces several fundamental challenges. First, because candidate rules typically require joins across multiple relations, the combinatorial search space is prohibitively large; indeed, finding the minimal join queries for a given collection of labeled examples is known to be exponential [55]. Second, the induced rules must remain concise while generalizing effectively to unseen data [55]. Since real-world databases are often noisy and incomplete, balancing simplicity and effectiveness is highly non-trivial: overly simplistic rules easily introduce false positives, whereas overly complex rules risk overfitting the training data [45, 53]. Finally, limited labeled data further exacerbates these trade-offs by biasing the learner toward underfitted, overly general rules. When underfitting is inevitable, designing robust mechanisms to still exploit these rules while ensuring accurate and reliable entity resolution remains an important open problem.

Our Approach. In this paper, we present a framework for learning declarative rules for collective ER based on *Learning from Failures* (LFF) [18], a prominent Inductive Logic Programming (ILP)

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

paradigm for relational learning [16, 48]. Given a learning input consisting of schema information, a database instance, and labelled examples, an LFF learner operates via a generate–test–constrain loop. It iteratively generates candidate rules, evaluates them against labeled examples, and derives constraints from failed rules; defined as candidates that either fail to cover positive examples or incorrectly entail negative ones. Guided by a subsumption ordering that compares rule generality, these derived constraints efficiently eliminate structurally related, underperforming candidates (both overly specific and overly general), thereby pruning massive search spaces. To address the trade-off between simplicity and effectiveness , we define optimal ER rulesets to minimise rule size while maximising coverage of positive examples. Inspired by [33], we formulate the search for such a ruleset as a Maximum Satisfiability (MaxSAT) problem. Furthermore, inspired by [9, 57], we adopt a recursive dynamic semantics for rule application, where derived merges are propagated iteratively to resolve interdependent relations. Crucially, this dynamic semantics naturally incorporates semantic integrity constraints (specifically, denial constraints), which eliminate inconsistent merges. This constraint enforcement substantially improves the quality of the resolution outcome, making underfitted rules useful even when only limited labelled data is available.

Contributions. We summarise our contributions as follows:

- We define a declarative ER rule language with tailored syntactic restrictions that reduce structural complexity. To support collective ER and semantic consistency, we adopt a recursive dynamic semantics for rule application that enforces denial constraints (DCs), providing declarative ASP encodings to compute the resulting solutions.
- We formulate collective ER rule learning under the LFF paradigm, introducing a formal subsumption ordering that captures rule generality. We prove this ordering preserves semantic monotonicity, where more general rules consistently yield a superset of merges, enabling effective search-space pruning.
- We introduce a levelwise learning algorithm that leverages a meta-level ASP program to generate candidate rules, using the subsumption ordering to derive pruning constraints. We propose an optimisation strategies realised by a MaxSAT encoding, to learn an optimal ER ruleset that best generalises the labelled examples while minimising its size.
- We evaluate our implementation on both pairwise and multi-relational datasets, demonstrating competitive accuracy and practical runtimes. The results show that multi-relational learning significantly outperforms pairwise baselines. Furthermore, by leveraging DCs, our framework compensates for underfitting when trained on a mere 20% of the data, recovering performance close to that achieved using the full training set.

2 PRELIMINARIES

A (named) schema $\mathcal{S}$ is a finite set of *relation symbols*, where each $R \in \mathcal{S}$ has an associated arity n and a list of *attributes* $(A_1, \ldots, A_n)$, each having a *domain*, denoted, $\mathsf{dom}(A)$. We write R/n to denote that R has arity n, and $R[A_1, \ldots, A_n]$ to indicate that $(A_1, \ldots, A_n)$ are its attributes. We call $(A_1, \ldots, A_n)$ the *sort of* R, denoted *sort*(R). A *database (instance)* D *over a schema* $\mathcal{S}$ assigns to each n-ary relation symbol $R \in \mathcal{S}$ a finite n-ary relation, R^D, over a fixed infinite set of constants C. One can view a database over $\mathcal{S}$ as a finite set of *facts* of the form $R(c_1, \ldots, c_n)$, where $(c_1, \ldots, c_n)$ is a tuple of constants from C of the same arity as R and compatible with its sort. We assume w.lo.g. that each tuple has an *id attribute* that identifies it. The *active domain* of D, denoted $\mathsf{adom}(D)$, is the set of all constants occurring in D.

Tuple Relational Calculus. We use tuple relational calculus syntax [1] to define ER rules. This will be convenient to express correlation among attributes from multiple relations in a given schema. Given a schema $\mathcal{S}$, an *atom* over $\mathcal{S}$ is either (i) a *relation atom* $R(t)$, where $R \in \mathcal{S}$, and t is a tuple variable (ii) a *comparison atom* $s[A]\ \Theta\ t[B]$, where s and t are tuple variables, A and B are attributes on which s and t are respectively defined, and $\Theta \in \{=, \approx\}$ is a *comparison operator*; attributes A, B are required to be *compatible (w.r.t.* Θ*)*, that is they have domains whose members can be compared by Θ. The operator $\approx$ is a binary relation with fixed semantics as a similarity relation on attribute values. We assume pairs of compatible attributes of the form (A, A') are provided by the schema.

A *conjunctive tuple relational calculus expression* is of the form: $\exists \vec{y}.\varphi(\vec{x}, \vec{y})$, where $\varphi(\vec{x})$ is a conjunction of atoms, and $\vec{x}$ and $\vec{y}$ are disjoint lists of tuple variables. Following safety conditions in the tuple relational calculus, we assume that every tuple variable t occurring in a comparison atom also occurs in some atom $R(t)$ in φ, with $R \in \mathcal{S}$. Further, for every atom $s[B_1] \approx t[B_2]$ in φ, (B_1, B_2) are compatible w.r.t. $\approx$, i.e., $\mathsf{dom}(B_1) \times \mathsf{dom}(B_2) \subseteq \approx$ [44].

Let D be a database over a schema $\mathcal{S}$. A *valuation* h of tuple variables of φ (in D) is a mapping that instantiates every tuple variable t in φ, with a tuple $\vec{c}$ in a relation in D. A valuation h satisfies a relation atom $R(v)$ iff $h(v) \in R^D$, and it satisfies comparison atom $s[B_1]\ \Theta\ t[B_2]$ iff $c_1\ \Theta\ c_2$ holds, where c_1 and c_2 are the B_1 and B_2 attributes of $h(s)$ and $h(t)$, resp. Satisfaction of an expression is defined by standard first-order logic semantics. The answers to an expression $\exists \vec{y}.\varphi(\vec{x}, \vec{y})$ over D, denoted as $\mathsf{ans}(\varphi(\vec{x}))$ is the set of tuples $\vec{c} = (c_1, \ldots, c_n) \in \mathsf{adom}(D)$, of the same size as $\vec{x}$, such that $\varphi(\vec{c})$ (the result of replacing each x_i by c_i) is satisfied in D.

Denial Constraints. We also consider Denial Constraints (DCs) [7] of the form $\bot \leftarrow \exists \vec{x}.\varphi(\vec{x})$, where $\varphi(\vec{x})$ is a finite conjunction of relation and comparison atoms, as well as inequality atoms $x[A] \neq x'[B]$. A denial constraint is satisfied in a database D if there is no valuation of its variables such that φ is satisfied.

Answer Set Programming (ASP): Syntax and Semantics. ASP is a declarative problem-solving paradigm rooted in logic programming and non-monotonic reasoning [29, 42]. It is particularly well-suited to address complex combinatorial search problems. In ASP, a problem is represented as a set of logical rules (a logic program), and the solutions correspond to the *answer sets* of that program.

A normal program is a finite set of rules of the form

$$H :- \ B_1, \ldots, B_m, \mathit{not}\ B_{m+1}, \ldots, \mathit{not}\ B_n$$

where H and each B_j are atoms. A rule with no head atom is called a *constraint*.

The semantics of an ASP program is defined in terms of the *stable model semantics*. The semantics is evaluated over the *Herbrand base of the ground program*, which is the set of all possible variable-free atoms. For a positive ground program Π (i.e., a program without negated body atoms), there exists a unique minimal model of Π,

called its *answer set*. For a ground program Π containing negated body atoms, the semantics is defined using the *reduct of* Π *with respect to (a candidate answer set)* M, denoted Π^M, and obtained by applying the following steps to Π: (i) remove all rules that contain a negative literal `not B` with $B \in M$; and (ii) from the remaining rules, remove all negative literals. The resulting program, Π^M, is positive. A set of ground atoms M is an *answer set* of Π if M coincides with the $\subseteq$-minimal model of Π^M. The semantics extends to programs with variables via *grounding*. Let $gr(\Pi)$ denote the set of all ground instances of rules in Π obtained using the constants occurring in Π. A set of ground atoms M is an answer set of Π if it is an answer set of $gr(\Pi)$. We write $SM(\Pi)$ for the set of all answer sets of Π.

3 ENTITY RESOLUTION RULES

In this section, we define the syntax, and semantics of ER rules. In particular, we distinguish between static and dynamic semantics: under the former, rules are evaluated once during learning for efficiency, whereas under the latter, rules are applied recursively during entity resolution to improve resolution quality. We then present ASP encodings for computing the resolution outcomes under the two semantics (Sections 3.1 and 3.2).

Syntax. An *ER rule* r *over a schema* $\mathcal{S}$ has the form

$$x[A] = y[B] \leftarrow \varphi(x, y)$$

where $\varphi(x, y)$ is a conjunctive tuple relational calculus expression and x, y are its free variables. We call the atom $x[A] = y[B]$ the *head* of the rule, denoted $head(r)$, and φ its *body*, denoted $body(r)$. For convenience, and slightly abusing notation, we shall conflate $body(r)$ with the set of atoms occurring in φ. We write $var(r)$ to denote the set of tuple variables in r.

Syntactic Restriction. We impose a syntactic restriction on ER rules to ensure that comparison atoms in rule bodies are relevant to merges. To make this condition more precise, we associate with each ER rule an undirected graph.

Let r be an ER rule. The *connectivity graph associated to* r is the undirected graph $G_r = \langle V, L\rangle$ with $V = var(r)$, and with L the set of edges $\{(s, t)\}$ such that there is a comparison atom in $body(r)$ where s and t occur. Let $V_h = \{x, y\}$ be the head variables in r. We say that r is *biconnected* if the following conditions hold:

- G_r is connected;
- for every rule variable $z \notin V_h$, there exist two distinct paths π, π' in G_r connecting t with x and y, respectively, and such that π and π' overlap only on z.

Semantics. Entity resolution can be formulated as the task of discovering pairs of syntactically distinct database constants that refer to the same entity (often called *merges*).

We now define the semantics of ER rules. Applying a set of ER rules to a database derives a set of entity merges. In our framework, we distinguish between two rule application modalities: *static semantics*, where merges are evaluated once over a fixed (static) database, and *dynamic semantics*, where merges are computed iteratively over the database induced by prior resolutions. This dynamic approach allows new rules that were not applicable in the original database to become applicable. Aligning with established ER frameworks that decouple rule discovery from application [38, 53], both modalities serve a strategic purpose in our framework. We employ static semantics during the learning phase to ensure computational efficiency, while reserving dynamic semantics for the application phase to deliver higher-quality solutions. Following the LACE framework for ER [9], we define a *solution for* (D, Σ), given an $\mathcal{S}$-database D and a set of ER rules Σ, as an equivalence relation over $\mathsf{adom}(D)$. For a binary relation $S \subseteq \mathsf{adom}(D) \times \mathsf{adom}(D)$, we denote by $\mathsf{EqRel}(S, D)$ the least equivalence relation E over $\mathsf{adom}(D)$ that contains S.

Example 1. *Consider the ER rules* r_1, r_2 *over the schema* $\mathcal{S}_{\text{mo}}$ *presented in Figure 1. The intended semantics of* r_1 *is that if two movies have similar titles and agree on both genre and year, then their identifiers should be merged. Rule* r_2 *is a multi-relational rule stating that two movie tuples have the same identifier (mid) whenever they have similar titles, and are associated with alias tuples having the same alternative title (aka) in the same region.*

For a rule $r = x[A] = y[A'] \leftarrow \varphi(x, y)$ and a given database D, we use $r(D)$ to denote the set $\{(c_A, c_{A'}) \mid (\vec{c_x}, \vec{c_y}) \in \mathsf{ans}(\varphi)\}$, where c_A (resp. $c_{A'}$) is the constant occurring as attribute A (resp. A') in $\vec{c}$.

Static Semantics. Given an $\mathcal{S}$-database D and a set of ER rules $\Sigma = \{r_1, \ldots, r_n\}$ over $\mathcal{S}$, the *static solution* for (D, Σ) is $E_{\text{s}} = \mathsf{EqRel}(S, D)$ where $S = \bigcup_{i=1}^{n} r_i(D)$.

Dynamic Semantics. Given a database D and an equivalence relation E over $\mathsf{adom}(D)$, the *database induced* by D and E, denoted D_E, is obtained by replacing each constant c with a fixed representative $\bar{c}$ of its E-equivalence class. Let $\varepsilon : \mathsf{adom}(D) \to \mathsf{adom}(D_E)$ be the mapping that takes each $c \in \mathsf{adom}(D)$ to $\bar{c} \in \mathsf{adom}(D_E)$. We extend this mapping naturally to tuples. For a tuple of constants $tp = (c_1, \cdots c_k)$, we write $\bar{\varepsilon}(tp)$ to denote the tuple $(\varepsilon(c_1), \ldots, \varepsilon(c_k))$.

We call E a *candidate solution* for (D, Σ) if it satisfies one of the two conditions:

(1) $E = \mathsf{EqRel}(\emptyset, D)$;
(2) $E = \mathsf{EqRel}(E' \cup \{(c_1, c_2)\}, D)$, where E' is a candidate solution for (D, Σ) and $(c_1, c_2) \in r(D_{E'}) \setminus E'$ for some $r \in \Sigma$.

Thus, candidate solutions can be regarded as a set of merges obtained by iteratively applying rules in Σ on a database induced by previous merges. We write $\mathsf{cand}(D, \Sigma)$ to denote the set of candidate solutions for (D, Σ). The *maximal solution* for (D, Σ) is the $\subseteq$-maximal candidate solution in $\mathsf{cand}(D, \Sigma)$.

In the presence of DCs, we are interested in consistent solutions. Given a set Δ of DCs, a *consistent solution* for (D, Σ, Δ) is a candidate solution E for (D, Σ) that satisfies Δ and such that there is no $E' \in \mathsf{cand}(D, \Sigma)$ with $E \subsetneq E'$ and $(D, E') \models \Delta$. We use $\mathsf{ConSol}(D, \Sigma, \Delta)$ to denote the set of consistent (candidate) solutions for (D, Σ, Δ).

Example 2. *Let* $\Sigma = \{r_1, r_3, r_4\}$ *in Figure 1, and* $e_1 = (m_1, m_2)$, $e_2 = (d_1, d_2)$, *and* $e_3 = (m_3, m_4)$. *For brevity, symmetric and reflexive pairs are omitted from all solutions. The static and maximal solutions for* (D_{mo}, Σ) *are* $E_{\text{s}} = \{e_2, e_3\}$ *and* $E_{\text{m}} = \{e_1, e_2, e_3\}$, *respectively. Observe that* e_1 *appears only in the maximal solution. Indeed, due to the addition of* d_1 *and* d_2, e_1 *can now be obtained by applying* r_4.

In addition, let $\Delta = \{\rho\}$, *where*

$$\rho : \bot \leftarrow alias(x), alias(y), x[\mathsf{aka}] \neq y[\mathsf{aka}],$$
$$x[\mathsf{mid}, \mathsf{region}] = y[\mathsf{mid}, \mathsf{region}]$$

A consistent solution for $(D_{\text{mo}}, \Sigma, \Delta)$ *is* $E_{\text{c}} = \{e_1, e_2\}$, *where* e_1 *is eliminated from* E_{m} *due to violation of* ρ.

Movie

mid	title	genre	year	director
m_1	The Thing	horror	1982	d_1
m_2	The Thing.	Horror	-	d_2
m_3	The Adventures of Tintin	Cartoon	1976	-
m_4	Adventure of Tintin	Cartoon	1976	-

Director

did	name	gender	year
d_1	John Carpenter	male	1932
d_2	J. Carpenter	male	1932

Alias

mid	aka	region
m_3	Le Petit Vingtième	FR
m_4	Moulinsart-Hollywood	FR

$r_1 : x[\text{mid}] = y[\text{mid}] \leftarrow \mathit{movie}(x), \mathit{movie}(y),$
$x[\text{title}] \approx y[\text{title}], x[\text{genre, year}] = y[\text{genre, year}]$

$r_2 : x[\text{mid}] = y[\text{mid}] \leftarrow \mathit{movie}(x), \mathit{movie}(y), \mathit{alias}(w), \mathit{alias}(z),$
$x[\text{title}] \approx y[\text{title}], x[\text{mid}] = w[\text{mid}], y[\text{mid}] = z[\text{mid}],$
$w[\text{aka, region}] = z[\text{aka, region}]$

$r_3 : x[\text{did}] = y[\text{did}] \leftarrow \mathit{director}(x), \mathit{director}(y)$
$x[\text{name}] \approx y[\text{name}], x[\text{gender, year}] = y[\text{gender, year}]$

$r_4 : x[\text{mid}] = y[\text{mid}] \leftarrow \mathit{movie}(x), \mathit{movie}(y),$
$x[\text{title}] \approx y[\text{title}], x[\text{director}] = y[\text{director}]$

Figure 1: A schema $\mathcal{S}_{\text{mo}}$, an $\mathcal{S}_{\text{mo}}$-database D_{mo} and ER rules defined over $\mathcal{S}_{\text{mo}}$.

3.1 ASP Encoding for Static Solutions

Let $\mathcal{S}$ be a schema, D an $\mathcal{S}$-database, and Σ a set of ER rules over $\mathcal{S}$. We define an ASP encoding $\Pi_s(D, \Sigma)$, whose answer sets correspond to the static solution for (D, Σ). For each tuple $(c_1, \ldots, c_n) \in R^D$ with $R(A_1, \ldots, A_n) \in \mathcal{S}$, we introduce a set of ASP facts Π_D as follows: $\mathtt{r(t)}$ where $\mathtt{t}$ is a unique constant identifying $(c_1, \ldots, c_n)$; $\mathtt{att(t, a_i, c_i)}$, where $\mathtt{c_i}$ and $\mathtt{a_i}$ are constants representing c_i and A_i, respectively. Each $r \in \Sigma$ is encoded as an ASP rule σ_r as follows: (i) Each relation atom $R(x) \in \mathit{body}(r)$ is encoded as an ASP atom $\mathtt{r(X)}$ in σ_r. (ii) For the rule head $x[A] = y[B]$ of r where x, y have range R, R', σ_r includes head $\mathtt{eqo(V_a, V_b)}$, and its body includes $\mathtt{att(X, a, V_a)}$, $\mathtt{att(Y, b, V_b)}$. (iii) For each equivalence class C of terms induced by join atoms in $\mathit{body}(r)$, introduce a fresh variable $\mathtt{V_C}$. For every join atom $x[A] = y[B]$ whose terms are from the same equivalence class, body of σ_r contains a pair of $\mathtt{att}$-atoms that share this variable in their third argument, i.e., $\mathtt{att(X, a, V_C)}$ and $\mathtt{att(Y, b, V_C)}$. (iv) For each similarity atom $x[A] \approx y[B] \in \mathit{body}(r)$, body of σ_r contains a pair of $\mathtt{att}$-atoms $\mathtt{att(X, a, V_a)}$ and $\mathtt{att(Y, b, V_b)}$, as well as $\mathtt{sim(V_a, V_b)}$. Finally, adding the following rules to $\Pi_s(D, \Sigma)$ to enforce that $\mathtt{eqo/2}$ is an equivalence relation:

```
eqo(X,Y):- eqo(Y,X).  eqo(X,Y):- eqo(X,Z),eqo(Z,Y).
```

Example 3. *Let $\Sigma = \{r_1, r_3, r_4\}$ and consider D_{mo} in Figure 1. An ASP encoding for r_1 in $\Pi_s(D_{\text{mo}}, \Sigma)$ is:*

```
eqo(X,Y):- movie(T1),movie(T2),att(T1,m,X),att(T2,m,Y),
        att(T1,g,V3),att(T2,g,V3),t(T1,r,V4),att(T2,r,V4),
        att(T1,t,V1),att(T2,t,V2),sim(V1,V2).
```

where m, g, r, t *are the constants representing attributes* mid, genre, region *and* title, *respectively.*

Proposition 1. *Let $\mathcal{S}$ be a schema, D an $\mathcal{S}$-database, and Σ a set of ER rules over $\mathcal{S}$. Then (c, c') is in the static solution for (D, Σ) iff $\mathtt{eqo(c, c')} \in M$, for some answer set M of $\Pi_s(D, \Sigma)$.*

3.2 ASP Encoding for Dynamic Solutions

We adapt the encoding of a static solution to compute maximal and consistent solutions in the presence of denial constraints. Let $\Pi_s(D, \Sigma)$ be an ASP encoding of the static solution for (D, Σ). Given a set of DCs Δ over $\mathcal{S}$, we construct ASP encodings for the maximal solution for (D, Σ) and the consistent solutions for (D, Σ, Δ), denoted by $\Pi_d(D, \Sigma)$ and $\Pi_d(D, (\Sigma, \Delta))$, respectively.

Encoding for Max. Solutions. The program $\Pi_d(D, \Sigma)$ is obtained from $\Pi_s(D, \Sigma)$ by applying the following transformations: (i) For every attribute A appearing in the head of a rule in Σ, and every attribute A' that is compatible with A , add into $\Pi_s(D, \Sigma)$ the rule $\mathtt{eqo(X, X):- att(T, A^*, X)}$, for $\mathtt{A^* = A}$ or $\mathtt{A'}$. (ii) For every rule in the resulting program, if a variable $\mathtt{V_i}$ occurs in two distinct $\mathtt{att}$-atoms in its body, replace one occurrence of $\mathtt{V_i}$ by a fresh variable $\mathtt{V'_i}$ and add $\mathtt{eqo(V_i, V'_i)}$ to the body. The static (resp. maximal) solution for (D, Σ) corresponds to the projection of the answer sets of $\Pi_s(D, \Sigma)$ (resp. $\Pi_d(D, \Sigma)$) onto the predicate $\mathtt{eqo/2}$.

Encoding for Consistent Solutions. Given DCs Δ, $\Pi_d(D, (\Sigma, \Delta))$ is obtained from $\Pi_d(D, \Sigma)$ by applying the following transformations: (i) For each encoding of an ER rule, transform it into a choice rule[1] by replacing every head atom $\mathtt{eqo(X, Y)}$ with $\{\mathtt{eqo(X, Y)}\}$, indicating that $\mathtt{eqo(X, Y)}$ may or may not be derived. (ii) For each $d \in \Delta$, construct an ASP constraint and add it to $\Pi_d(D, \Sigma)$ by first translating d into an ASP constraint as for ER rules (cf. Step (ii)above), with an empty head and excluding inequality atoms. Then, for each inequality atom over attributes A, B in d, if either A or B appears in the head of a rule in Σ, add $\mathtt{not\ eqo(V_a, V_b)}$ to the constraint body, where $\mathtt{V_a}$ and $\mathtt{V_b}$ are the variables of the corresponding $\mathtt{att}$-atoms; otherwise, append the inequality atom $\mathtt{V_a \neq V_b}$.

Example 4. *Continuing Example 3. Let $\Delta = \{\rho\}$ (ρ as defined in Example 2). An ASP encoding of r_2 in $\Pi_d(D_{\text{mo}}, (\Sigma, \Delta))$ in Figure 1 is:*

```
{eqo(X,Y)}:- movie(T1),movie(T2),alias(T3),alias(T4),
    att(T3,g,V3),att(T4,g,V3),attt(T3,a,V4),att(T4,a,V4),
    att(T1,m,X),att(T2,m,Y),att(T3,m,V5),att(T4,m,V6),
    eqo(X,V5),eqo(Y,V6),att(T1,t,V1),att(T2,t,V2),sim(V1,V2).
```

where m, g, a, t *are the constants representing attributes* mid, genre, aka *and* title, *respectively.*

An ASP encoding of the DC ρ in $\Pi_d(D_{\text{mo}}, (\Sigma, \Delta))$ is:

```
:- alias(T1),alias(T2),att(T1,m,X),att(T2,m,Y),eqo(X,Y),
att(T1,a,V1),att(T2,a,V2),att(T1,r,V3),att(T2,r,V3),V1!=V2.
```

We note that, in the presence of Δ, $\Pi_d(D, (\Sigma, \Delta))$ may admit multiple answer sets due to the interaction between choice rules and constraints.

[1] Choice rules are special syntactic sugar available in ASP.

Proposition 2. *Let $\mathcal{S}$ be a schema, D be an $\mathcal{S}$-database and (Σ, Δ) be a pair of ER ruleset and DCs over $\mathcal{S}$. Then (i) (c, c') is in the maximal solution for (D, Σ) iff $\mathsf{eqo}(\mathsf{c}, \mathsf{c}') \in M$ for some answer set M of $\Pi_d(D, \Sigma)$. (ii) A consistent solution $E \in \mathsf{ConSol}(D, \Sigma, \Delta)$ iff $E = \{(c, c') \mid \mathsf{eqo}(\mathsf{c}, \mathsf{c}') \in M\}$ where M is an answer set of the ASP encoding $\Pi_d(D, (\Sigma, \Delta))$.*

4 PROBLEM SETTING

We start by introducing key notions underlying our ER learning problem. Let us fix a schema $\mathcal{S}$ and a database instance D over $\mathcal{S}$.

Target Space. A *target* over $\mathcal{S}$ is a tuple of the form (R, A, R', A'), where $R, R' \in \mathcal{S}$, and A and A' are compatible attributes of R and R', resp. Given a target $\tau = (R, A, R', A')$, the *target space* identified by $\mathcal{S}$ and τ, denoted $\Gamma(\mathcal{S}, \tau)$, is the set of all biconnected ER rules defined over $\mathcal{S}$ of the form $x[A] = y[A'] \leftarrow \varphi$, where x and y are distinct variables and both $R(x)$, $R'(y)$ occur in φ.

Learning Instance. A *(rule) learning instance* is a tuple $\langle \mathcal{S}, \tau, D, \mathsf{Ex}^+, \mathsf{Ex}^- \rangle$, where $\mathcal{S}$ is a schema, D and $\tau = (R, A, R', A')$ are respectively a database and a target over $\mathcal{S}$; and Ex^+ and Ex^- are respectively non-empty sets of *positive* and *negative* examples of the form $(c, c') \in \mathsf{dom}(A) \cap \mathsf{adom}(D) \times \mathsf{dom}(A') \cap \mathsf{adom}(D)$.

Following the LFF paradigm [18], one can define the following notions. Given $\mathcal{I} = \langle \mathcal{S}, \tau, D, \mathsf{Ex}^+, \mathsf{Ex}^- \rangle$, a rule $r \in \Gamma(\mathcal{S}, \tau)$ is *supported (in $\mathcal{I}$)* if $r(D) \cap \mathsf{Ex}^+ \neq \emptyset$; it is *consistent* if $r(D) \cap \mathsf{Ex}^- = \emptyset$.

Real-world databases are often noisy and inconsistent [7]; consequently, requiring strict rule consistency can be overly restrictive. Therefore, unlike [18], we aim to learn rules that are *approximately consistent* while maintaining sufficient support. This relaxation not only accommodates data noise but also mitigates the risk of overfitting during the rule learning process [43]. Ultimately, solving a learning instance $\mathcal{I}$ amounts to discovering a ruleset that yields an approximate solution to the underlying ER problem.

Approximations. Let $\mathcal{I} = \langle \mathcal{S}, \tau, D, \mathsf{Ex}^+, \mathsf{Ex}^- \rangle$ be a learning instance, $r \in \Gamma(\mathcal{S}, \tau)$ and $\theta, \delta \in [0, 1)$. We call r *θ-consistent* if $|r(D) \cap \mathsf{Ex}^-| \leq |\mathsf{Ex}^-| \cdot \theta$. Furthermore, r is *δ-pure* if r is supported and $\frac{|r(D) \cap \mathsf{Ex}^-|}{|r(D) \cap \mathsf{Ex}^+|} \leq \delta$. Note that r is consistent iff it is 0-consistent. We say that r is *(θ, δ)-consistent* if it is θ-consistent and δ-pure; otherwise, it is inconsistent. r is called a *failure* if it is either not supported or inconsistent. A ruleset $\Sigma \subseteq \Gamma(\mathcal{S}, \tau)$ is a *(θ, δ)-approximation (to $\mathcal{I}$)*, if every $r \in \Sigma$ is supported, θ-consistent and δ-pure in $\mathcal{I}$.

Optimal Approximations. Instead of considering arbitrary approximations, we are interested in those that cover as many positive examples as possible, while remaining as succinct and general as possible. We will measure the quality of a ruleset in terms of its size. Let $\mathit{size}(\Sigma)$ denote the sum of the number of body atoms of every rule in Σ. Given a learning instance $\mathcal{I} = \langle \mathcal{S}, \tau, D, \mathsf{Ex}^+, \mathsf{Ex}^- \rangle$, a (θ, δ)-approximation Σ is *optimal* if is satisfies the following two conditions: (i) there exists no other (θ, δ)-approximation Σ' such that $|S \cap \mathsf{Ex}^+| < |S' \cap \mathsf{Ex}^+|$, where S and S' are the static solutions for (D, Σ) and (D, Σ'), respectively; and (ii) for every other (θ, δ)-approximation Σ'' such that the static solution for $(D, \Sigma'') = S$, it holds that $\mathit{size}(\Sigma) \leq \mathit{size}(\Sigma'')$.

5 SUBSUMPTION ORDER

In the previous section, we formalised our learning setting and defined the target space of ER from which a ruleset is learned. Note that even with our syntactic restrictions, this target space remains infinite. To make its exploration feasible, this section introduces a subsumption order that structures the target space to guide efficient pruning.

Rule Closure. To compare rules within a given target space, we must account for implicit comparison atoms in rule bodies that are not explicitly present. More precisely, for any rule r, we define its *closure*, denoted $\mathsf{cl}(r)$, as the rule obtained by extending $\mathit{body}(r)$ to transitively close its join atoms and add all implied similarity atoms. Formally, $\mathit{body}(\mathsf{cl}(r))$ is the smallest set of atoms that contains $\mathit{body}(r)$. Furthermore, whenever two join atoms $s[A] = t[B]$ and $t[B] = u[C]$ are in $\mathit{body}(\mathsf{cl}(r))$, the transitively implied join $s[A] = u[C]$ must also belong to $\mathit{body}(\mathsf{cl}(r))$. Additionally, any join atom $s[A] = t[B] \in \mathit{body}(\mathsf{cl}(r))$ automatically implies the presence of its corresponding similarity atom $s[A] \approx t[B] \in \mathit{body}(\mathsf{cl}(r))$. Finally, the closure satisfies symmetry, meaning that if $s[A] \,\Theta\, t[B] \in \mathit{body}(\mathsf{cl}(r))$, then its symmetric counterpart $t[B] \,\Theta\, s[A]$ must also be contained in $\mathit{body}(\mathsf{cl}(r))$ for any operator $\Theta \in \{=, \approx\}$.

Rule Subsumption. Let r and r' be two ER rules from the same target space. We say that r *subsumes* r', denoted $r \succeq r'$, if there exists a mapping $h : \mathit{var}(r) \to \mathit{var}(r')$ such that $\mathit{head}(rh) = \mathit{head}(r')$ and $\mathit{body}(rh) \subseteq \mathit{body}(\mathsf{cl}(r'))$, where rh is the rule obtained by replacing every occurrence of $t \in \mathit{var}(r)$ with $h(t)$. We say that r an r' are *$\succeq$-equivalent*, if $r \succeq r'$, and $r' \succeq r$,

Rule Generalisations and Specialisations. For any rules r, r' in a given target space $\Gamma(\mathcal{S}, \tau)$, we say that r is a *generalisation* of r', and dually that r' is a *specialisation* of r, if $r \succeq r'$. This notion extends naturally to sets of rules; for a finite set $\gamma \subset \Gamma(\mathcal{S}, \tau)$, a rule $r \in \Gamma(\mathcal{S}, \tau)$ is a *generalisation of γ* if $r \succeq r'$ for each $r' \in \gamma$. Furthermore, we say that r is a *least generalisation* (LG) of γ if $r'' \succeq r$ for every generalisation r'' of γ. Dually, a rule $r \in \Gamma(\mathcal{S}, \tau)$ is called a *specialisation of γ* if $r' \succeq r$ for all $r' \in \gamma$, and it constitutes a *greatest specialisation* (GS) of γ if $r \succeq r''$ for every specialisation r'' of γ.

An ER rule r in a target space $\Gamma(\mathcal{S}, \tau)$ is *minimal*, if there does not exist $r' \in \Gamma(\mathcal{S}, \tau)$ s.t. r' is $\succeq$-equivalent to r, and $|\mathit{body}(r')| < |\mathit{body}(r)|$.

Example 5. *Consider the following ER rules r, r' and r'' in the target space $\Gamma(\{R[A, B, C]\}, (R, A, R, A))$. We have that (i) $r \succ r'$, therefore r is a generalisation of r', and dually r' is a specialisation of r. (ii) r and r'' are $\succeq$-equivalent, and r is minimal but r'' is not.*

$r : x[A] = y[A] \leftarrow R(x), R(y), x[B] = y[B].$

$r' : x[A] = y[A] \leftarrow R(x), R(y), x[B] = y[B], x[C] \approx y[C].$

$r'' : x[A] = y[A] \leftarrow R(x), R(y), R(z), x[B] = z[B], y[B] = z[B].$

Lemma 1. *Let $\mathcal{S}$ be a schema and $\Gamma(\mathcal{S}, \tau)$ be the target space identified by τ. For any non-empty finite subset $\gamma \subseteq \Gamma(\mathcal{S}, \tau)$, there exists a rule $r_l \in \Gamma(\mathcal{S}, \tau)$ such that r_l is a minimal GS of γ.*

Lemma 2. *Let $\mathcal{S}$ be a schema and $\Gamma(\mathcal{S}, \tau)$ be the target space identified by τ. For any non-empty finite subset $\gamma \subseteq \Gamma(\mathcal{S}, \tau)$, there exists rule $r_u \in \Gamma(\mathcal{S}, \tau)$ such that r_u is a minimal LG of γ.*

We can use the subsumption relation $\succeq$ to define a partial order on (sets) of rules in a given target space. Since *subsumption equivalence* is an equivalence relation, we can order the set of all equivalence classes induced by $\equiv$. More precisely, let $\Gamma(\mathcal{S}, \tau)/_{\equiv}$ be the set of equivalence classes of $\Gamma(\mathcal{S}, \tau)$ induced by $\equiv$. We extend

the subsumption relation to equivalence classes in a natural way: for all $[\rho], [\rho'] \in \Gamma(\mathcal{S},\tau)/\equiv$, $[\rho] \succeq [\rho']$ if there is some $r \in [\rho]$ such that $r \succeq r'$, for some (all) $r' \in [\rho']$.

Theorem 1. *$\langle \Gamma(\mathcal{S},\tau)/_{=}, \succeq \rangle$ is a lattice.*

Cores and Minimal Rule Candidates. When exploring a target space during learning, we might encounter multiple candidate rules that are $\succeq$-equivalent. We are interested in learning those with the most concise representation (in the target space). We say that $r \in [\rho]$ is a *core* (in $[\rho]$) if its *minimal*. The following, observation provides the basis for an algorithmic computation of minimal candidate rules.

Lemma 3. *Let $r \in \Gamma(\mathcal{S},\tau)$. There exists a rule $r' \in \Gamma(\mathcal{S},\tau)$ and a mapping from $var(r)$ to $var(r')$ such that r and r' are $\succeq$-equivalent and r' is a core.*

Next, we show that it follows that given two ER rules over a schema, the strictly more general one guarantees to produce a larger set of merges for every database instance over the schema.

Theorem 2. *Let $\mathcal{S}$ be a schema and τ a target over $\mathcal{S}$. For every r, $r' \in \Gamma(\mathcal{S},\tau)$ and every $\mathcal{S}$-database D, it holds that $r \succeq r'$ iff $r'(D) \subseteq r(D)$.*

It then follows, that for two $\succeq$-equivalent rules, $r(D) = r'(D)$, for every database D. One implication of this result is that preferring minimal equivalent candidate rules during the learning process does not affect their desired properties (e.g, consistency).

Notably, containment of merge-sets also holds on the induced database if one rule subsumes the other:

Proposition 3. *Let $\mathcal{S}$ be a schema and τ a target over $\mathcal{S}$. For every $r, r' \in \Gamma(\mathcal{S},\tau)$, and every $\mathcal{S}$-database. If $r \succeq r'$, then $r'(D_E) \subseteq r(D_E)$ for any equivalence relation E over* $\mathsf{adom}(D)$.

Finally, we can use Theorem 2 to obtain a pruning strategy while searching for candidate rules in the target space.

Corollary 1. *Given a learning input $\langle \mathcal{S}, \tau, D, \mathsf{Ex}^+, \mathsf{Ex}^- \rangle$, ER rules $r, r' \in \Gamma(\mathcal{S},\tau)$, then the following hold:*

- *If r is inconsistent, then every generalisation r' of r is also inconsistent.*
- *For every $r, r' \in \Gamma(\mathcal{S},\tau)$ such that $r \succeq r'$, we have that $r'(D) \cap \mathsf{Ex}^+ \subseteq r(D) \cap \mathsf{Ex}^+$.*

6 LEARNING ER RULES FROM FAILURE

We present a learning algorithm based on the Learning from Failures (LFF) paradigm [18]. Generally, an LFF learner iteratively: (i) generates a candidate rule from the target space; (ii) evaluates it against the training examples; and (iii) if the rule violates the success criteria (e.g., θ-consistent and δ-pure) , derives constraints to prune its generalisations or specialisations. We first detail our meta-level ASP program for rule generation (Section 6.1) and explain how failed rules are converted into pruning constraints (Section 6.2). Finally, we present the LeCER algorithm to compute an optimal ruleset, if one exists (Section 6.3).

6.1 ASP Program for ER Rule Generation

Inspired by ASP meta-programming techniques [30, 36], we employ a meta-level ASP program for rule generation. Under this approach, we encode the input schema and target attributes as ASP facts. The meta-program then generates answer sets, each of which encodes a valid ER rule within the target space. More precisely, for a given learning instance $\mathcal{I} = \langle \mathcal{S}, \tau, D, \mathsf{Ex}^+, \mathsf{Ex}^- \rangle$, we define a meta-level ASP program $\Pi_{\mathsf{meta}}(\mathcal{S},\tau)$ that computes the target space for $(\mathcal{S},\tau)$. It consists of a set of facts encoding $\mathcal{S}$ and τ from $\mathcal{I}$, together with a set of ASP rules for rule generation and constraints imposing syntactic restrictions (e.g., biconnectivity). An answer set of the program contains a set of *meta-atoms* corresponding to an ER rule at the meta-level.

```
1  % Initialising head-relevant atoms
2  head(eqo,2,(0,1)). head_var(0). head_var(1).
3  {body(att,3,(V+2,A,V)): vars(3,(V+2,A,V)), head_var(V), target(_,A,V)}=2.
4  body_var(V) :- body(R,A,VS), vars(V,VS,P), not attr(V).
5
6  body(R,1,2) :- body(att,3,(2,A,0)), target(R,A,_).
7  body(R,1,3) :- body(att,3,(3,A,1)), target(R,A,_).
8
9  % Expand body with relation atom
10 {body(R,1,T)} :- body(R',1,T'), has_sort(R,A), has_sort(R',A'), comp_a(A,A'),
11                 vars(3,(T,A,_)), T!=T'.
12 % Expand body with attribute atom
13 {body(att,3,(T,A,V))} :- body(R,1,T), vars(3,(T,A,V)).
14 % Expand body with similarity atom
15 {body(sim,2,(V,V'))} :- body(R,1,T), body(R',1,T'), body(att,3,(T,A,V)),
16                        body(att,3,(T',A',V')), comp_a(A,A'), V!=V'.
17
18 % Variable combinations
19 num_vars(n).
20 var(0..N-1) :- num_vars(N).
21 vars(2,(V1,V2)) :- var(V1), var(V2).
22 vars(3,(V1,V2,V3)) :- var(V1), attr(V2), var(V3).
23
24 % Size aggregate
25 size(N) :- #sum{1:body(_,1,T); 1:body(att,3,(T,_,V)), body(att,3,(T',_,V)),
26          T!=T'; 1:body(sim,2,(V,V'))}==N.
```

Figure 2: ASP encodings for ER rule generation

Schema Declaration. We use the predicates `schema/1`, `attr/1`, and `r/2` to represent the schema name, attribute names, and relations, respectively. To represent $(\mathcal{S},\tau)$ from $\mathcal{I}$, we include the following facts: $\mathtt{schema(s)}$, where `s` is a unique constant denoting the name of the schema; $\mathtt{r(s,r_i)}$ for each relation $R_i \in \mathcal{S}$; $\mathtt{attr(a_j)}$ for each attribute $A_j \in \mathsf{A}$; $\mathtt{has\_sort(r_i,a_j)}$ for each attribute $A_j \in \mathit{sort}(R_i)$; $\mathtt{target(r_i,a_j,p)}$ for each attribute A_j of R_i that occurs in the target τ, where $\mathtt{p} \in \{0,1\}$, indicating the order of an attribute in τ (e.g., for pairwise matching). In addition, to declare compatible attributes, we include a fact $\mathtt{comp\_a(a,a')}$ for each pair of compatible attributes A, A', together with the following rules:

```
comp_a(X,X):- attr(X).  comp_a(X,Y):- comp_a(Y,X).
comp_a(X,Y):- comp_a(X,Z), comp_a(Z,Y).
```

ER Rule Generation. The ASP meta-program for generating ER rules is detailed in Figure 2. Its core objective is to compute answer sets consisting of *meta-atoms*, `head/3` and `body/3`, where arguments `(P,A,V)` denote the predicate name, arity, and variable schema of the target ER rule. Lines 18–22 define the variable name space, bounding the available variables and generating valid combinations of variables and attributes. Lines 1–7 initialize the rule head and its base bindings: Line 2 establishes the head predicate `eqo/2`, while subsequent rules generate the initial `att`-atoms from

`target` facts and derive the corresponding relation atoms that bind their tuple IDs. The remaining rules incrementally expand the rule body. Line 10 introduces new relation atoms that share a compatible attribute with an existing relation but use a distinct tuple variable. Line 13 extends these relations with their respective `att`-atoms. Line 15 adds similarity atoms (`sim`) between compatible attribute pairs present in the body. Finally, Line 25 uses a `#sum` aggregate to compute the total rule size by tallying its relation, join, and similarity atoms.

Syntactic Constraints. To refine the over-approximated search space generated by Figure 2, $\Pi_{\text{meta}}(\mathcal{S}, \tau)$ also includes a set of ASP constraints that prune invalid candidate rules. Specifically, they eliminate comparisons between incompatible attributes and enforce structural restrictions: biconnectivity (cf Section 3) among tuple variables. To guarantee biconnectivity, the constraints impose a lexicographical order over variable names to form a directed graph flowing from a designated "tail" variable toward the head variables. This structural enforcement ensures every newly added tuple variable forms distinct non-diverging paths to each of the head variables thereby imposing the biconnectivity restriction. Full encodings and further details are deferred to Appendix C.

We show that $\Pi_{\text{meta}}(\mathcal{S}, \tau)$ is a sound encoding of $\Gamma(\mathcal{S}, \tau)$, and complete for all representable ER rules in $\Gamma(\mathcal{S}, \tau)$ using the specified number of variable names:

Proposition 4. *Given a learning instance $\langle \mathcal{S}, \tau, D, \mathsf{Ex}^+, \mathsf{Ex}^- \rangle$, every answer set of $\Pi_{\text{meta}}(\mathcal{S}, \tau)$ (via its meta atoms) corresponds to an ER rule in $\Gamma(\mathcal{S}, \tau)$. Moreover, for a given number n of distinct variable names, every $r \in \Gamma(\mathcal{S}, \tau)$ with $|body(r)| \leq \frac{n+2}{2}$ is represented by an answer set of $\Pi_{\text{meta}}(\mathcal{S}, \tau)$.*

Note that the completeness bound on rule size is sufficient but not necessary: ER rules whose size exceeds the bound may also be represented by answer sets of $\Pi_{\text{meta}}(\mathcal{S}, \tau)$.

6.2 Rule Space Pruning

We describe how failures in a learning instance are transformed into constraints that are added to $\Pi_{\text{meta}}(\mathcal{S}, \tau)$ to prune their generalisations and specialisations.

Generalisation Constraints (GCs). To prune generalisations, we add GCs to $\Pi_{\text{meta}}(\mathcal{S}, \tau)$. Specifically, given the meta-atoms representing an ER rule r, we construct a GC to eliminate all $\succeq$-equivalent rules of r by: (i) replacing every constant `c` denoting a variable name in the second or third position of a `head` or `body` atom with a fresh variable V_c; and (ii) appending a ground atom `size(n)` to ensure the constraint triggers only when the generated meta-atoms correspond to an ER rule of size n.

Example 6. *Let $\mathcal{S} = \{R[A, B, C]\}$ be a schema and $\tau = (R, A, R, A)$ be a target over $\mathcal{S}$. The following meta-atoms generated by $\Pi_{\text{meta}}(\mathcal{S}, \tau)$:*

```
head(eqo, 2, (0, 1)). body(r, 1, 2). body(r, 1, 3).
body(att, 3, (2, a, 0)). body(att, 3, (2, a, 0)).
body(att, 3, (2, b, 4)). body(att, 3, (2, b, 4)).
```

corresponding to the ER rule $r : x[A] = y[A] \leftarrow R(x), R(y), x[B] = y[B]$ whose size is 3, and whose ASP encoding is

```
eqo(V0, V1) :- r(V2), r(V3), att(V2, a, V0), att(V2, a, V1),
               att(V3, b, V4), att(V3, b, V4).
```

Algorithm 1 LeCER

```
1:  Input: learning instance ⟨S, τ, D, Ex+, Ex−⟩, approximation parameters (θ, δ), maximal rule size m
2:  Output: Optimal ruleset Σ
3:  Σ, P, C := ∅, i := 1, exhausted := false;
4:  M := an empty Hash Map;
5:  while i ≤ m do
6:      (r, exhausted) := Generate(C, i)
7:      if ∃r' ∈ M such that r' ⪰ r and M[r'] = true then
8:          C := C ∪ Constrain(true, true, r)
9:          continue
10:     else if ∃r' ∈ M such that r ⪰ r' and M[r'] = false then
11:         C := C ∪ Constrain(true, false, r)
12:         continue
13:     end if
14:     (T, F) := Test(Ex+, Ex−, D, r)
15:     consistent := IsConsistent(θ, δ, T, F, Ex+, Ex−)
16:     supported := (|T| > 0)
17:     covers_new := (|T \ P| > 0)
18:     if T = Ex+ and consistent then
19:         return {r}
20:     else if supported and consistent then
21:         P := P ∪ T, Σ := Σ ∪ {r}
22:         if P = Ex+ then
23:             return Combine(Σ, D, Ex+)
24:         end if
25:     end if
26:     C := C ∪ Constrain(supported, consistent, r)
27:     if supported and covers_new then
28:         r := ComputeCore(r)
29:         M[r] := consistent
30:     end if
31:     if exhausted then
32:         i := i + 1
33:     end if
34: end while
35: return Combine(Σ, D, Ex+)
```

A GC to prune all $\succeq$-equivalent generalisations of r is

```
:- size(3), head(eqo, 2, (V0, V1)), body(r, 1, V2), body(r, 1, V3),
   body(att, 3, (V2, a, V0)), body(att, 3, (V3, a, V1)),
   body(att, 3, (V2, b, V4)), body(att, 3, (V3, b, V4)), V.
```

where $\mathcal{V}$ is the conjunction $\mathsf{V0}! = \mathsf{V1}, ..., \mathsf{V3}! = \mathsf{V4}, ..., \mathsf{V0}! = \mathsf{V4}$, enforcing inequalities between each pair of distinct variables.

We also introduce a set of GCs to eliminate generalisations with weaker comparisons, i.e., rules obtained by replacing join atoms with similarity atoms. Each such GC is constructed by selecting m out of the n join atoms in r and converting them into similarity atoms (accounting for symmetries), where $m \in [1, n]$.

Example 7. *Continuing Example 6, a set of GCs eliminate generalisations with weaker comparisons includes:*

```
:- head(eqo, 2, (V0, V1)), body(r, 1, V2), body(r, 1, V3),
   body(att, 3, (V2, a, V0)), body(att, 3, (V3, a, V1)),
   body(att, 3, (V2, b, V4)), body(att, 3, (V3, b, V5)),
   body(sim, 2, (V4, V5)), size(3), V.
```

Similarly, $\mathcal{V}$ is the list of conjunction enforces inequalities for each pair of distinct variables. A symmetric GC of above is obtained by replacing $\texttt{body}(\texttt{sim}, 2, (\texttt{V}_4, \texttt{V}_5))$ *with* $\texttt{body}(\texttt{sim}, 2, (\texttt{V}_5, \texttt{V}_4))$.

Specialisation Constraints (SCs). SCs are constructed similarly to GCs, with two key differences. First, before constructing an SC, we compute the core (see Algorithm 3 in Appendix A) of the rule whose specialisations are to be pruned. The meta-atoms in the SC body are then derived directly from this core, omitting the `size` atom. Second, comparisons in the core are strengthened rather than weakened. Specifically, each SC is generated by selecting m out of the n similarity atoms in r and converting them into join atoms, where $m \in [1, n]$.

Example 8. *Consider again $\mathcal{S}$ and τ from Example 6, the meta-atoms represent the following rule r'*

$$x[A] = y[A] \leftarrow R(x), R(y), R(z), x[B] = y[B], \\ x[B] = z[B], x[C] = z[C], x[C] \approx y[C].$$

The core of r' is

$$x[A] = y[A] \leftarrow R(x), R(y), x[B] = y[B], x[C] \approx y[C].$$

The set of SCs to prune specialisations of r' includes a SC built from meta atoms of its core

$$\texttt{:- head(eqo, 2, (V}_0\texttt{, V}_1\texttt{)), body(r, 1, V}_2\texttt{), body(r, 1, V}_3\texttt{),} \\ \texttt{body(att, 3, (V}_2\texttt{, a, V}_0\texttt{)), body(att, 3, (V}_3\texttt{, a, V}_1\texttt{)),} \\ \texttt{body(att, 3, (V}_2\texttt{, b, V}_4\texttt{)), body(att, 3, (V}_3\texttt{, b, V}_5\texttt{)),} \\ \texttt{body(sim, 2, (V}_4\texttt{, V}_5\texttt{)), } \mathcal{V}.$$

and those by removing $\texttt{body}(\texttt{sim}, 2, (\texttt{V}_4, \texttt{V}_5))$ *and unifying* $\texttt{V}_4, \texttt{V}_5$.

We show that the encodings of GCs and SCs are sound.

Proposition 5. *Let $\mathcal{S}$ be a schema, $\Gamma(\mathcal{S}, \tau)$ the target space defined by $(\mathcal{S}, \tau)$, and $\Pi_{\text{meta}}(\mathcal{S}, \tau)$ the corresponding generation program. For every $M_r \in SM(\Pi_{\text{meta}}(\mathcal{S}, \tau))$ representing a rule $r \in \Gamma(\mathcal{S}, \tau)$, a generalisation (resp. specialisation) constraint constructed from M_r prunes generalisations (resp. specialisations) of r, and never prunes any rule $r' \in \Gamma(\mathcal{S}, \tau)$ such that $r \succ r'$ (resp. $r' \succ r$).*

6.3 Learning Algorithm

We present our level-wise rule learning algorithm, LeCER (Alg. 1). Given a learning instance, a maximum rule size, and optional approximation parameters, LeCER iteratively searches the rule space and returns an optimal ruleset if one exists.

LFF Loop. The algorithm initialises three empty sets, Σ, C, and P, to store: (i) candidate ER rules, (ii) SCs and GCs learned from previously evaluated ER rules, and (iii) positive examples covered by Σ. Additionally, a hash map $\mathcal{M}$ maps the core of each supported ER rule to its consistency status. Following [18], LeCER iteratively executes the standard LFF loop across three stages: generation (Lines 5–12), testing (Lines 13–16), and constraint generation (Lines 7–8 and 26). Each iteration begins by generating an ER rule r of size i via the ASP program described in Section 6.1. Unlike [18], LeCER performs a subsumption check against previously evaluated rules prior to testing r (Lines 7–12) If a more general, (θ, δ)-consistent ER rule has already been found, we prune the specialisations of r to favour more general candidates. Conversely, if r is more general than a previously evaluated, supported but inconsistent rule, then r is also inconsistent by Corollary 1, allowing us to prune its generalisations. Following the subsumption check, the Constrain function generates pruning constraints based on the status of r (Section 6.2): SCs if r is (θ, δ)-consistent; GCs if r is supported but inconsistent; and both SCs and GCs if r is unsupported and inconsistent.

During the testing stage, the generated rule r is evaluated on the examples using its ASP encoding for static solutions, yielding the sets of positive and negative examples covered by r (Line 14). If r is (θ, δ)-consistent and covers all positive examples, the algorithm immediately returns the singleton ruleset $\{r\}$ (Lines 18–19). Otherwise, if r is supported and consistent, it is added to Σ, and P is expanded with the positive examples covered by r (Line 21). Subsequently, the algorithm invokes Constrain to derive pruning constraints from r.

The loop terminates when a complete ruleset is found or the maximum search size is reached. The algorithm then invokes the Combine function to identify an optimal subset of the candidate set Σ that maximises positive example coverage whilst minimising ruleset size (Lines 23 and 35).

Optimisation. To search for an optimal ruleset, the Combine function leverages machinery from the Popper system [17, 33], formulating the task as a Maximum Satisfiability (MaxSAT) problem [4]. In MaxSAT, a *literal* represents a propositional variable (p.v.) or its negation [32], and a *clause* is a disjunction of literals. Given a set of hard clauses and a set of weighted soft clauses, the objective is to find a truth assignment that satisfies all hard clauses whilst minimising the cumulative penalty incurred by any falsified soft clauses.

Let Σ denote the learned ruleset input to Combine (cf. Lines 17 and 29 of Algorithm 1), and let $\Sigma^* \subseteq \Sigma$ denote the output ruleset. Following [18, 33], for each candidate rule $r \in \Sigma$, a p.v. p_r indicates whether r is included in Σ^*. Similarly, for each example $e^+ \in \texttt{Ex}^+$, a p.v. c_{e^+} indicates whether e^+ is covered by the static solution for (D, Σ). In contrast to [18, 33], we also introduce additional p.v.s to account for the transitive derivation of positive examples. Specifically, let $\tilde{\epsilon}(e^+, \texttt{Ex}^+)$ denote the pair of positive examples in E that transitively induce e^+. For each positive example $e^+ \in \texttt{Ex}^+$, we introduce a p.v. t_{e^+} indicating whether e^+ is transitively induced by other positive examples. We then enforce this by adding the following hard clause:

$$c_{e^+} \rightarrow \bigvee_{e^+ \in r(D), r \in \Sigma} p_r \ \vee \ t_{e^+}, \tag{1}$$

ensuring that every covered positive example is either derived directly by a selected rule $r \in \Sigma^*$ or obtained through transitivity. To capture transitive derivations, for every pair of distinct examples $e_1^+, e_2^+ \in \epsilon(e^+, \texttt{Ex}^+)$, we add the hard clause: $c_{e_1^+} \wedge c_{e_2^+} \rightarrow t_{e^+}$.

To search for an optimal solution, the following soft clauses are added: (i) for each candidate rule $r \in \Sigma$, the soft clause $\neg p_r$ with penalty $size(r)$, discouraging solutions with large sizes; and (ii) for each positive example $e^+ \in \texttt{Ex}^+$, the soft clause c_{e^+} with penalty 1, encouraging coverage of positive examples. The resulting hard and soft clauses are then passed to a MaxSAT solver, whose optimal truth assignment corresponds to an optimised ruleset.

Theorem 3. *Given a learning instance $\mathcal{I}$, a maximal rule size m, and approximation parameters (θ, δ), Algorithm LeCER returns an optimal (approximation) ruleset in $\mathcal{I}$ if one exists.*

Name	Type	#Rec	#Rel	#At	#Dup
Dblp	Pairwise	66,879	2	10	5,347
Cora	Pairwise	1,879	1	17	64,578
Imdb	Relational	11,424	3	15	3,666
Poke	Relational	37,782	3	26	702

Table 1: Dataset Statistics. #-columns represent the number of records, relations, attributes and duplicates, respectively.

Note that although LeCER evaluates ER rules under the static semantics, their generality is preserved over induced databases (cf. Proposition 3), allowing them to be correctly applied under the dynamic semantics.

7 EXPERIMENTAL STUDY

We evaluate LeCER on real-world ER datasets, spanning both pairwise and multi-relational scenarios. Specifically, our evaluation aims to answer the following four questions: (i) How does our method compare against existing state-of-the-art approaches? (ii) What is the effect of enabling multi-relational learning? (iii) How robust is our approach when varying the amount of training data? (iv) Can DCs help with under-fit solutions?

Experimental setting

Implementation and environment. LeCER 's implementation is built upon the open-source LFF system, Popper [18]. While Algorithm 1 and subsumption order functionalities are written in Python, the ASP program for rule generation are implemented using the Clingo API. We conducted all experiments on a workstation powered by a single core of an AMD Ryzen Threadripper 5965WX processor running at 3.8GHz.

Datasets. We evaluate LeCER using two pairwise matching datasets widely adopted in the literature: DBLP-Scholar [38, 53] and CORA, along with two multi-relation datasets previously used for collective ER evaluation [19, 57]: (i) a subset of the IMDB movie dataset; (ii) a Pokémon dataset, sampled from a comprehensive Pokémon species database. For brevity, we refer to these datasets as *Dblp, Cora, Imdb*, and *Poke*, respectively. Their key statistics are summarized in Table 1. Following the experimental setup in [38, 53], we employ an 80/20 train-test split.

Negative sampling. While positive examples are obtained directly from the ground-truth matches in the datasets, we sample an equal number of negative examples from the non-matching entity pairs in the training data. To avoid uninformative negatives, such as entirely unrelated tuple pairs that offer no supervisory signal, we only retain negative pairs where at least one corresponding attribute pair satisfies either $\approx$ or $=$, following [53]. Furthermore, we aim to maintain a near-uniform distribution of these attribute conditions across the selected negative pairs.

Similarity measures and metrics. We compute the syntactic similarities of constants based on their data types [5] using standard metrics such as Levenshtein distance and Jaccard similarity. To evaluate the effectiveness of the discovered learning solutions, we use the metrics [37] of Precision (P), Recall (R), and F1-Score (F_1).

Selection of parameters. To select θ and δ, we use 20% of the training data from each dataset as a validation set and perform a grid search over $\theta \in \{0.5, 0.8, 1, 1.5, 2, 3, 4\}$ and $\delta \in \{1, 0.5, 0.25, 0.15, 0.08, 0.05, 0.025\}$ to maximize the validation F_1-score. This yields $(\theta, \delta) = (0.5, 0.25)$ for *Dblp*, $(2, 0.15)$ for *Cora*, $(0.1, 1)$ for *Imdb*, and $(0.5, 0.1)$ for *Poke*. For optimal performance, we cap the number of variable names at 8 and 12, and the maximal rule size at 10 and 12, for the pairwise and multi-relational datasets, respectively.

Experimental results

Exp-1: Pairwise Datasets. We evaluate LeCER on *Dblp* and *Cora* against two primary baselines: (i) RuSyn [53], a program synthesis approach that learns ER rules represented as General Boolean Formulae; and (ii) MDedup [38], a system that discovers matching dependencies (MDs) for duplicate detection.[2] For both baselines, we report the results of their best-performing configuration directly from [53] and [38].[3] For MDedup, while the core MD discovery process is unsupervised, it typically generates a prohibitively large number of candidate dependencies. To address this, the original pipeline utilizes labeled examples to either train a classifier for MD selection or select high-performing MDs directly. We compare LeCER against this example-selected pipeline, as it directly mirrors our learning setting and represents the empirical upper-bound performance reported for MDedup [38]. Following the experimental protocols in [38, 53], we evaluate LeCER using 5-fold cross-validation to mitigate sampling bias. For each dataset, records are randomly partitioned into five equal folds. Each fold is held out once for testing while the remaining four folds serve as the training set. All reported metrics are averaged over the five runs.

Effectiveness. We report the average quality metrics (P, R, and F_1) alongside the execution time (t) for the pairwise datasets in Table 2. Overall, the rules learned by LeCER are highly competitive, achieving the second-best F_1-score on both *Dblp* and *Cora*. While RuSyn obtains the highest F_1-scores, outperforming LeCER by 6.8% and 5.1% respectively, both LeCER and RuSyn substantially outperform MDedup. In particular, LeCER yields an F_1 improvement over MDedup of 85.7% on *Dblp* and 14.1% on *Cora*. In terms of precision, the example-selected MDs from MDedup tend to be more conservative, surpassing LeCER by approximately 10% on *Dblp* and 13% on *Cora*. However, this high precision comes at the cost of significantly lower coverage: LeCER achieves a substantial recall advantage of 81.9% and 32% over MDedup on *Dblp* and *Cora*, respectively.

Exp-2: Multi-relational Datasets. To evaluate the benefits of multi-relational learning, we compare our approach in a pairwise configuration (LeCER/pw) versus the multi-relational configuration (LeCER) on the two multi-relation datasets.

Effectiveness. As shown in Table 3, leveraging multi-relational learning consistently improves the F_1-scores achieved by LeCER across both multi-relational datasets. Specifically, on *Poke*, incorporating multi-relational rules yields a substantial 58% increase in F_1-score, whereas LeCER/pw exhibits severely degraded performance in the

[2]We note in passing that while approaches for discovering entity-enhancing rules [23–25] are conceptually relevant to ER rule learning no ER evaluation results have been reported for the discovered rules.

[3]The source code for RuSyn is not publicly available. For MDedup unresolved software dependency issues prevented us from locally reproducing the experiments.

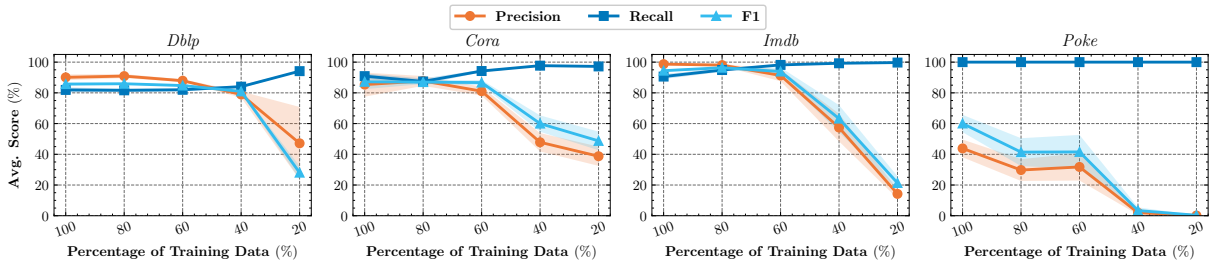

Figure 3: Impact of the amount of training data on effectiveness. Shaded regions indicate variance.

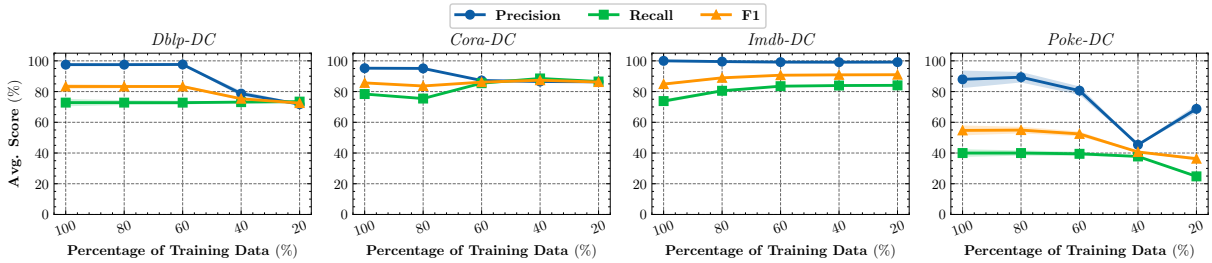

Figure 4: Impact of the amount of training data on effectiveness when DCs are presented. Shaded regions indicate variance.

Method	Data	F_1 (%)	P (%)	R (%)	t (s)	$\lvert\Sigma\rvert$
RuSyn *	*Dblp*	**92.5**	N/a	N/a	1,747	N/a
MDedup *		0	1	0	17,643	N/a
LeCER		85.7 ±2	90 ±4.2	81.9 ±3	68 ±3.4	4.8 ±0.4
LeCER $^{\Delta}$		85.7 ±2	90 ±4.2	81.9 ±3	52 ±3.7	10.2 ±0.8
RuSyn *	*Cora*	**92.2**	N/a	N/a	2,064	N/a
MDedup *		73	98	58	97	N/a
LeCER		87.1 ±10	85.3 ±17	90.8 ±3	125.4 ±27	4 ±0.7
LeCER $^{\Delta}$		68.2 ±13	57.5 ±20	90.8 ±6	107.6 ±12	15.2 ±6.5

Table 2: Results on pairwise datasets. Methods marked with * report the results from their original papers. LeCER $^{\Delta}$ denotes the ablation variant without combine stage.

Method	Data	F_1 (%)	P (%)	R (%)	t (s)	$\lvert\Sigma\rvert$
LeCER/pw	*Imdb*	84.9 ±3.3	99.5 ±1	74.1 ±4.8	329 ±368	1.8 ±0.4
LeCER		94.4 ±1.7	98.6 ±1.8	90.6 ±3.5	914.6 ±689	3.2 ±0.8
LeCER $^{\Delta}$		94.4 ±1.7	98.6 ±1.8	90.6 ±3.5	984.4 ±579	4.8 ±1.7
LeCER/pw	*Poke*	1.9 ±3	1 ±1.5	99.1 ±1.8	2,663 ±815	4 ±1.7
LeCER		60 ±12	43.8 ±12	100	1,782 ±421	1
LeCER $^{\Delta}$		60 ±12	43.8 ±12	100	1,782 ±421	1

Table 3: Results on multi-relation datasets. LeCER $^{\Delta}$ denotes the ablation variant without combine stage.

absence of relational context. In terms of precision and recall, the rules discovered by LeCER consistently outperform LeCER/pw in coverage, achieving a recall advantage of 16.5% on *Imdb* and 0.9% on *Poke*. This demonstrates that certain duplicates can only be identified when dependencies across related entities are evaluated jointly. Furthermore, while LeCER incurs a minor precision drop

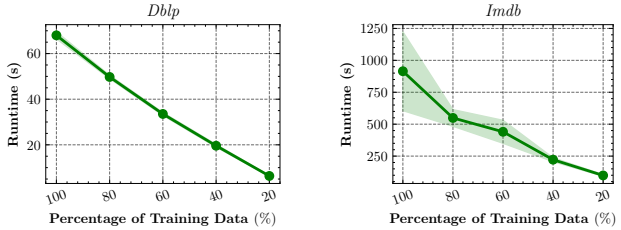

Figure 5: Impact of the amount of training data on efficiency. Shaded regions indicate variance.

of 0.9% on *Imdb*, it achieves a remarkable 42.8% precision gain on *Poke*. These results underscore that when duplicate identification hinges on collective information across multiple relations, multi-relational rules not only boost coverage but also significantly enhance matching accuracy.

Efficiency. In terms of execution time, LeCER is 9× slower than LeCER/pw on *Imdb*, but achieves a 1.4× speedup on *Poke*. An inspection of the discovered rule sets reveals that both configurations generate multiple rules on *Imdb*. In contrast, on *Poke*, LeCER/pw requires a disjunction of multiple rules, whereas LeCER converges to a single, concise multi-relational rule. These results demonstrate that while expanding the search space to multi-relational rules generally introduces additional computational overhead, discovering a highly expressive relational rule that tightly fits the data can trigger earlier termination of the learning algorithm (cf. Algorithm 1). We also observe that runtime variance is substantially higher on multi-relational datasets than on their pairwise counterparts. This divergence is driven by the fact that LeCER evaluates larger rules over multi-relational schemas, which leads to greater volatility in validation and evaluation costs during learning.

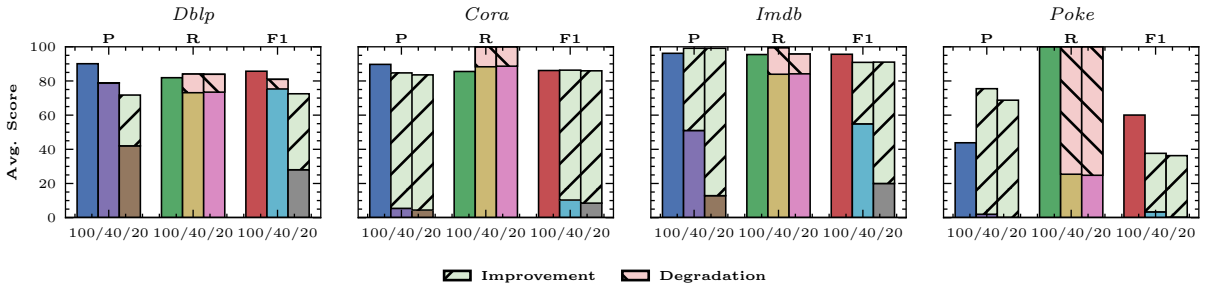

Figure 6: Impact of incorporating DCs to underfitting rules. Shaded bars are the improvements (green)/degradation (red) after incorporating DCs.

Exp-3: Optimisation. We conduct an ablation study to evaluate the impact of the Combine stage for optimisation, on both pairwise and multi-relational datasets.

without combine stage. The results are reported in Tables 2 and 3, where the configurations LeCER and LeCER$^{\Delta}$ denote the variants with and without the Combine stage, respectively. As illustrated, incorporating the Combine stage consistently reduces the total number of rules ($|\Sigma|$) in the discovered specifications without compromising matching accuracy. In particular, on *Cora*, which contains substantial duplication, the optimisation reduces the number of rules to nearly one quarter of the baseline specification size while simultaneously improving the F_1-score by 19%. In terms of execution time, the Combine stage introduces additional computational overhead, resulting in a 1.3× and 1.1× slowdown on *Dblp* and *Cora*, respectively. Conversely, on *Imdb*, the optimised variant achieves a lower average execution time. This anomalous result is driven by the high runtime variance inherent to the underlying multi-relational learning procedure; this specific outcome should therefore be interpreted as an effect of runtime variability rather than an algorithmic reduction in computational cost. Overall, these findings demonstrate the efficacy of the Combine stage in producing more compact specifications with enhanced generalisation, at the expense of a reasonable computational overhead.

To evaluate the sensitivity of our approach to training data volume, we varied the proportion of training data across $p \in \{100\%, 80\%, 60\%, 40\%, 20\%\}$. To mitigate sampling variance, for each of the four training folds and for every sub-sample size $p < 100\%$, we generated multiple random subsets. Specifically, we sampled 2, 3, 4, and 5 independent runs for 80%, 60%, 40%, and 20% of the training data, respectively.

Exp-4: Varying Training Data Volume.

Results. Figures 3 and 5 illustrate the impact of training data volume on effectiveness and efficiency, respectively. Regarding efficiency, *Cora* and *Poke* exhibit performance trends similar to those of *Dblp* and *Imdb*; we thus omit their curves for brevity and focus on the latter two. Across most datasets, the discovered rules achieve F_1-scores comparable to the full-training baseline even when utilising only 60% of the training data. Notably, on *Dblp*, performance remains robust and virtually unchanged with just 40% of the training data. Conversely, on *Poke*, the F_1-score drops considerably when the training set size decreases from 60% to 40%, highlighting a critical data threshold below which the quality of rule induction severely degrades.

When the training set is restricted to 20%, underfitting becomes pronounced across all datasets. Compared to the full-training baseline, the F_1-score decreases by approximately 50% and 40% on *Dblp* and *Cora*, respectively. On the multi-relational datasets, the rules induced from just 20% of the training data yield near-perfect recall but near-zero precision, indicating that the search converges to overly general hypotheses. These findings demonstrate that while LeCER is highly robust when trained on roughly half of the available data, severe scarcity of labeled examples (e.g., on *Poke* or under a 20% training split) makes the induction process susceptible to underfitting by biasedly favoring overly general rules.

Similar scaling trends are observed for training efficiency. For pairwise datasets like *Dblp*, execution time decreases smoothly, exhibiting a close-to-linear reduction as the training set size scales down. In contrast, multi-relational datasets exhibit significantly higher runtime variance, particularly when larger portions of the training data are utilized. This behaviour is inherently tied to the search space characteristics of multi-relational learning, which admits a much larger set of candidate expansions during the level-wise search. Consequently, slight variations in the training data can lead the algorithm down vastly different search trajectories before it converges to an optimal solution.

Exp-5: Incorporating DCs For each dataset and training data proportion $p \in \{100\%, 80\%, 60\%, 40\%, 20\%\}$, we augment the induced ER rule set with the dataset's functional dependencies formulated as DCs, subsequently evaluating the resulting consistent ER solutions. Because a learned rule set combined with DCs can yield multiple consistent solutions, we enumerate up to 50 such solutions [58] and report the average quality metrics for each proportion p across all datasets.

Results. Figure 4 illustrates the quality of the resulting ER solutions after incorporating DCs into the induced rule sets across varying training data volumes. Compared to the baseline results in Figure 3, incorporating DCs consistently enhances the reliability of the learned ER rules across all datasets and training proportions. This improvement is driven by a substantial increase in precision, albeit at the expense of a minor reduction in coverage. Consequently, the overall ER solutions exhibit significantly greater robustness, even in data-scarce scenarios where severe underfitting was previously observed in Figure 3.

To further isolate the impact of DCs, Figure 6 compares the solution quality for the underfitting regimes ($p = 40\%$ and $p = 20\%$) before and after integrating DCs, alongside the baseline obtained by training without DCs on the full dataset ($p = 100\%$). With the exception of $p = 40\%$ on *Dblp*, incorporating DCs substantially improves the F_1-score across all datasets for both low-data splits, bringing empirical performance close to the full-data baseline. This improvement is primarily driven by a significant boost in precision, accompanied by only negligible reductions in recall (except on *Poke*). Notably, on the multi-relational datasets, the precision achieved after incorporating DCs even surpasses that of the full-data baseline. These results demonstrate that integrity constraints are crucial for enhancing the consistency of ER solutions, particularly when rules must be induced from noisy data (e.g., *Cora*) or highly limited labeled examples.

Summary. Our empirical study yields four key takeaways. First, LeCER achieves highly competitive matching accuracy, securing the second-best F_1-score on pairwise datasets and substantially outperforming MDedup with up to an 85.7% F_1 improvement.Second, evaluating our system in a multi-relational configuration demonstrates the clear necessity of collective ER; incorporating relational context across related entities yields up to a 58% increase in F_1-score over the pairwise-only baseline (LeCER/pw). Third, while LeCER exhibits high robustness by maintaining baseline-comparable performance when trained on only 60% of the data, severe data scarcity (20% training volume) leads to pronounced underfitting and overly general hypotheses. Finally, augmenting these underfitted, low-data rules with DCs effectively mitigates this vulnerability by boosting precision, recovering ER quality close to the full-data baseline.

8 RELATED WORK

Declarative ER. Various declarative frameworks have been proposed for ER, ranging from Datalog-style rules [3] and Matching Dependencies (MDs) [21, 22, 26] to more expressive formalisms like REEs [27] and MRL [19], which enrich MDs with machine learning predicates and attribute comparisons [19]. Recently, the Lace framework [9, 10] and its ASP-based implementation ASPEn [57, 58] integrated hard and soft rules with denial constraints to provide high-accuracy, explainable collective ER. Similar to ASPEn [57], we use ASP to encode ER rules and adopt the global semantics of [9], where merged constants are treated as identical database-wide. This global perspective is ideal for resolving object-representing constants (e.g., entity IDs), whereas local semantics (which equate constants only within individual tuples) are better suited for merging attribute values [10, 20]. Under this semantics, our maximal solution forms an upper bound on the ER solutions in [57], while our consistent solutions correspond to ASPEn's maximal solutions restricted to soft rules and denial constraints. Crucially, unlike [57], combining our induced ER rules with denial constraints guarantees the existence of at least one consistent solution for any database instance.

Data Quality Rule Discovery. Considerable effort has been devoted to discovering data quality rules, such as DCs and FDs [15, 34, 43, 50]. Closer to our work are methods for learning MDs [38, 52] and REEs [23–25]. While we similarly employ a level-wise search over a lattice-structured rule space [34, 52], our framework differs in three fundamental aspects. First, we utilize a declarative meta-level ASP program for search-space generation and pruning. Second, unlike standard partial orders based on set containment over rule bodies, our subsumption ordering is tailored to multi-relational, collective ER rules, capturing the semantic generality of both join and similarity atoms. Third, existing MD and REE discovery techniques assume rule heads hold on supporting tuples, enabling unsupervised discovery. This assumption fails when resolving object constants (e.g., entity IDs), where duplicates are initially represented by syntactically distinct constants. In contrast, our framework leverages labeled examples to supervise the learner in inferring equivalences between these distinct object constants. The closest approach is [53], which synthesizes pairwise ER rules via program synthesis with examples; however, our framework uniquely supports learning collective ER rules.

Inductive Logic Programming (ILP). ILP provides robust theoretical foundations and efficient systems for relational learning [16, 18, 35, 40, 47, 48, 51]. While our framework is inspired by Popper, a state-of-the-art Learning from Failures engine [17, 18, 33], adapting it to collective ER requires three non-trivial extensions: (i) we design a novel meta-program, meta-constraints, and a subsumption ordering tailored to the dynamic semantics of collective ER rules; (ii) we perform eager, subsumption checking immediately after rule generation to prune the search space more aggressively than standard LFF; and (iii) we define a new optimization objective that natively handles noisy databases by computing optimal approximate solutions.

9 CONCLUSION

We introduced a novel LFF-based framework for discovering declarative collective ER rules. By formulating collective rule learning within an ILP paradigm, LeCER employs a meta-level ASP generator guided by a designated rule subsumption ordering to achieve effective search-space pruning. We also incorporated a MaxSAT-driven optimisation stage to synthesise concise, optimal rulesets that successfully maximise recall while minimising rule size. Our extensive empirical evaluation demonstrates that multi-relational rule learning significantly outperforms standard pairwise baselines. Crucially, our framework bridges the gap in low-data regimes; by augmenting induced rulesets with DCs, LeCER successfully mitigates pronounced underfitting when trained on a mere 20% of the data, recovering precision and accuracy close to full-training benchmarks. Ultimately, this work offers a robust, concise, and explainable declarative approach to scaling collective entity resolution on real-world, multi-relational databases.

We identify several promising directions for future work. First, exploring alternative learning paradigms, such as active learning, could further minimise the requirements for labelled examples [12]. Second, since semantic monotonicity holds for our definition of collective ER rules, our framework can naturally support incremental rule discovery and revision when new data arrives [2], a task well-suited for refinement operators in inductive logic programming [48]. Finally, we plan to extend our framework beyond resolving object constants to learn rules specifically for data value merges [10, 58], which would necessitate incorporating a local semantics.

REFERENCES

[1] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley.

[2] Yasser Altowim, Dmitri V. Kalashnikov, and Sharad Mehrotra. 2018. ProgressER: Adaptive Progressive Approach to Relational Entity Resolution. *ACM Trans. Knowl. Discov. Data* 12, 3 (2018), 33:1–33:45. https://doi.org/10.1145/3154410

[3] Arvind Arasu, Christopher Ré, and Dan Suciu. 2009. Large-Scale Deduplication with Constraints Using Dedupalog. In *Proceedings of the 25th International Conference on Data Engineering, ICDE 2009, March 29 2009 - April 2 2009, Shanghai, China*, Yannis E. Ioannidis, Dik Lun Lee, and Raymond T. Ng (Eds.). IEEE Computer Society, 952–963. https://doi.org/10.1109/ICDE.2009.43

[4] Fahiem Bacchus, Matti Järvisalo, and Ruben Martins. 2021. Maximum Satisfiability. In *Handbook of Satisfiability - Second Edition*, Armin Biere, Marijn Heule, Hans van Maaren, and Toby Walsh (Eds.). IOS Press, 929–991. https://doi.org/10.3233/FAIA201008

[5] Zeinab Bahmani, Leopoldo E. Bertossi, and Nikolaos Vasiloglou. 2017. ERBlox: Combining matching dependencies with machine learning for entity resolution. *Int. J. Approx. Reason.* 83 (2017), 118–141. https://doi.org/10.1016/j.ijar.2017.01.003

[6] Omar Benjelloun, Hector Garcia-Molina, David Menestrina, Qi Su, Steven Euijong Whang, and Jennifer Widom. 2009. Swoosh: a generic approach to entity resolution. *VLDB J.* 18, 1 (2009), 255–276. https://doi.org/10.1007/S00778-008-0098-X

[7] Leopoldo E. Bertossi. 2011. *Database Repairing and Consistent Query Answering*. Morgan & Claypool Publishers. https://doi.org/10.2200/S00379ED1V01Y201108DTM020

[8] Indrajit Bhattacharya and Lise Getoor. 2007. Collective entity resolution in relational data. *ACM Transactions on Knowledge Discovery from Data* 1, 1 (2007), 5.

[9] Meghyn Bienvenu, Gianluca Cima, and Víctor Gutiérrez-Basulto. 2022. LACE: A Logical Approach to Collective Entity Resolution. In *PODS '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*, Leonid Libkin and Pablo Barceló (Eds.). ACM, 379–391. https://doi.org/10.1145/3517804.3526233

[10] Meghyn Bienvenu, Gianluca Cima, Víctor Gutiérrez-Basulto, and Yazmín Ibáñez-García. 2023. Combining Global and Local Merges in Logic-based Entity Resolution. In *Proceedings of the 20th International Conference on Principles of Knowledge Representation and Reasoning, KR 2023, Rhodes, Greece, September 2-8, 2023*, Pierre Marquis, Tran Cao Son, and Gabriele Kern-Isberner (Eds.). 742–746. https://doi.org/10.24963/KR.2023/74

[11] Meghyn Bienvenu, Gianluca Cima, Víctor Gutiérrez-Basulto, Yazmín Ibáñez-García, and Zhiliang Xiang. 2025. Recent Advances in Logic-Based Entity Resolution. *SIGMOD Rec.* 54, 3 (2025), 7–21. https://doi.org/10.1145/3774303.3774305

[12] Angela Bonifati, Radu Ciucanu, and Slawek Staworko. 2014. Interactive Inference of Join Queries. In *Proceedings of the 17th International Conference on Extending Database Technology, EDBT 2014, Athens, Greece, March 24-28, 2014*, Sihem Amer-Yahia, Vassilis Christophides, Anastasios Kementsietsidis, Minos N. Garofalakis, Stratos Idreos, and Vincent Leroy (Eds.). OpenProceedings.org, 451–462. https://doi.org/10.5441/002/EDBT.2014.41

[13] Zhiyu Chen, Jing Ma, Xinlu Zhang, Nan Hao, An Yan, Armineh Nourbakhsh, Xianjun Yang, Julian J. McAuley, Linda Ruth Petzold, and William Yang Wang. 2024. A Survey on Large Language Models for Critical Societal Domains: Finance, Healthcare, and Law. *Trans. Mach. Learn. Res.* 2024 (2024). https://openreview.net/forum?id=upAWnMgpnH

[14] Vassilis Christophides, Vasilis Efthymiou, Themis Palpanas, George Papadakis, and Kostas Stefanidis. 2020. An Overview of End-to-End Entity Resolution for Big Data. *ACM Comput. Surv.* 53, 6, Article 127 (Dec. 2020), 42 pages. https://doi.org/10.1145/3418896

[15] Xu Chu, Ihab F. Ilyas, and Paolo Papotti. 2013. Discovering Denial Constraints. *Proc. VLDB Endow.* 6, 13 (2013), 1498–1509. https://doi.org/10.14778/2536258.2536262

[16] Andrew Cropper and Sebastijan Dumancic. 2022. Inductive Logic Programming At 30: A New Introduction. *J. Artif. Intell. Res.* 74 (2022), 765–850. https://doi.org/10.1613/JAIR.1.13507

[17] Andrew Cropper and Céline Hocquette. 2023. Learning Logic Programs by Combining Programs. In *ECAI 2023 - 26th European Conference on Artificial Intelligence, September 30 - October 4, 2023, Kraków, Poland - Including 12th Conference on Prestigious Applications of Intelligent Systems (PAIS 2023) (Frontiers in Artificial Intelligence and Applications)*, Kobi Gal, Ann Nowé, Grzegorz J. Nalepa, Roy Fairstein, and Roxana Radulescu (Eds.). IOS Press, 501–508. https://doi.org/10.3233/FAIA230309

[18] Andrew Cropper and Rolf Morel. 2021. Learning programs by learning from failures. *Mach. Learn.* 110, 4 (2021), 801–856. https://doi.org/10.1007/S10994-020-05934-Z

[19] Ting Deng, Wenfei Fan, Ping Lu, Xiaomeng Luo, Xiaoke Zhu, and Wanhe An. 2022. Deep and Collective Entity Resolution in Parallel. In *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*. IEEE, 2060–2072. https://doi.org/10.1109/ICDE53745.2022.00200

[20] Ronald Fagin, Phokion G. Kolaitis, Domenico Lembo, Lucian Popa, and Federico Scafoglieri. 2023. A Framework for Combining Entity Resolution and Query Answering in Knowledge Bases. In *Proceedings of the 20th International Conference on Principles of Knowledge Representation and Reasoning, KR 2023, Rhodes, Greece, September 2-8, 2023*, Pierre Marquis, Tran Cao Son, and Gabriele Kern-Isberner (Eds.). 229–239. https://doi.org/10.24963/KR.2023/23

[21] Wenfei Fan. 2008. Dependencies revisited for improving data quality. In *Proceedings of the Twenty-Seventh ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems, PODS 2008, June 9-11, 2008, Vancouver, BC, Canada*, Maurizio Lenzerini and Domenico Lembo (Eds.). ACM, 159–170. https://doi.org/10.1145/1376916.1376940

[22] Wenfei Fan and Floris Geerts. 2012. *Foundations of Data Quality Management*. Morgan & Claypool Publishers. https://doi.org/10.2200/S00439ED1V01Y201207DTM030

[23] Wenfei Fan, Ziyan Han, Yaoshu Wang, and Min Xie. 2022. Parallel Rule Discovery from Large Datasets by Sampling. In *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*, Zachary G. Ives, Angela Bonifati, and Amr El Abbadi (Eds.). ACM, 384–398. https://doi.org/10.1145/3514221.3526165

[24] Wenfei Fan, Ziyan Han, Yaoshu Wang, and Min Xie. 2023. Discovering Top-k Rules using Subjective and Objective Criteria. *Proc. ACM Manag. Data* 1, 1 (2023), 70:1–70:29. https://doi.org/10.1145/3588924

[25] Wenfei Fan, Ziyan Han, Min Xie, and Guangyi Zhang. 2024. Discovering Top-k Relevant and Diversified Rules. *Proc. ACM Manag. Data* 2, 4 (2024), 195:1–195:28. https://doi.org/10.1145/3677131

[26] Wenfei Fan, Xibei Jia, Jianzhong Li, and Shuai Ma. 2009. Reasoning about Record Matching Rules. *Proc. VLDB Endow.* 2, 1 (2009), 407–418. https://doi.org/10.14778/1687627.1687674

[27] Wenfei Fan, Ping Lu, and Chao Tian. 2020. Unifying logic rules and machine learning for entity enhancing. *Sci. China Inf. Sci.* 63, 7 (2020). https://doi.org/10.1007/S11432-020-2917-1

[28] Ivan P. Fellegi and Alan B. Sunter. 1969. A Theory for Record Linkage. *J. Amer. Statist. Assoc.* 64 (1969), 1183–1210. https://api.semanticscholar.org/CorpusID:17349112

[29] Martin Gebser, Roland Kaminski, Benjamin Kaufmann, and Torsten Schaub. 2012. *Answer Set Solving in Practice*. Morgan & Claypool Publishers.

[30] Martin Gebser, Roland Kaminski, and Torsten Schaub. 2011. Complex optimization in answer set programming. *Theory Pract. Log. Program.* 11, 4-5 (2011), 821–839.

[31] Lise Getoor and Ashwin Machanavajjhala. 2012. Entity Resolution: Theory, Practice & Open Challenges. *Proc. VLDB Endow.* 5, 12 (2012), 2018–2019. https://doi.org/10.14778/2367502.2367564

[32] Valentin Goranko. 2016. *Logic as a Tool - A Guide to Formal Logical Reasoning*. Wiley. http://eu.wiley.com/WileyCDA/WileyTitle/productCd-1118880005.html

[33] Céline Hocquette, Andreas Niskanen, Matti Järvisalo, and Andrew Cropper. 2024. Learning MDL Logic Programs from Noisy Data. In *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2014, February 20-27, 2024, Vancouver, Canada*, Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan (Eds.). AAAI Press, 10553–10561. https://doi.org/10.1609/AAAI.V38I9.28925

[34] Ykä Huhtala, Juha Kärkkäinen, Pasi Porkka, and Hannu Toivonen. 1999. TANE: An Efficient Algorithm for Discovering Functional and Approximate Dependencies. *Comput. J.* 42, 2 (1999), 100–111. https://doi.org/10.1093/COMJNL/42.2.100

[35] Katsumi Inoue, Tony Ribeiro, and Chiaki Sakama. 2014. Learning from interpretation transition. *Mach. Learn.* 94, 1 (2014), 51–79. https://doi.org/10.1007/S10994-013-5353-8

[36] Roland Kaminski, Javier Romero, Torsten Schaub, and Philipp Wanko. 2023. How to Build Your Own ASP-based System?! *Theory Pract. Log. Program.* 23, 1 (2023), 299–361.

[37] Hanna Köpcke, Andreas Thor, and Erhard Rahm. 2010. Evaluation of entity resolution approaches on real-world match problems. *Proc. VLDB Endow.* 3, 1 (2010), 484–493. https://doi.org/10.14778/1920841.1920904

[38] Ioannis Koumarelas, Thorsten Papenbrock, and Felix Naumann. 2020. MDedup: duplicate detection with matching dependencies. *Proc. VLDB Endow.* 13, 5 (Jan. 2020), 712–725. https://doi.org/10.14778/3377369.3377379

[39] Shrinu Kushagra, Hemant Saxena, Ihab F. Ilyas, and Shai Ben-David. 2019. A Semi-Supervised Framework of Clustering Selection for De-Duplication. In *35th IEEE International Conference on Data Engineering, ICDE 2019, Macao, China, April 8-11, 2019*. IEEE, 208–219. https://doi.org/10.1109/ICDE.2019.00027

[40] Mark Law, Alessandra Russo, and Krysia Broda. 2014. Inductive Learning of Answer Set Programs. In *Logics in Artificial Intelligence - 14th European Conference, JELIA 2014, Funchal, Madeira, Portugal, September 24-26, 2014. Proceedings (Lecture Notes in Computer Science)*, Eduardo Fermé and João Leite (Eds.), Vol. 8761. Springer, 311–325. https://doi.org/10.1007/978-3-319-11558-0_22

[41] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. 2020. Deep Entity Matching with Pre-Trained Language Models. *Proc. VLDB Endow.* 14, 1 (2020), 50–60. https://doi.org/10.14778/3421424.3421431

[42] Vladimir Lifschitz. 2019. *Answer Set Programming*. Springer.
[43] Ester Livshits, Alireza Heidari, Ihab F. Ilyas, and Benny Kimelfeld. 2020. Approximate Denial Constraints. *CoRR* abs/2005.08540 (2020). arXiv:2005.08540 https://arxiv.org/abs/2005.08540
[44] David Maier. 1983. *The Theory of Relational Databases*. Computer Science Press. http://web.cecs.pdx.edu/%7Emaier/TheoryBook/TRD.html
[45] Albert Martin, Eduardo C. de Almeida, Oscar Romero, and Anna Queralt. 2025. How and Why False Denial Constraints are Discovered. *Proc. VLDB Endow.* 18, 10 (2025), 3477–3489. https://doi.org/10.14778/3748191.3748209
[46] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, and Vijay Raghavendra. 2018. Deep Learning for Entity Matching: A Design Space Exploration. In *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10-15, 2018*, Gautam Das, Christopher M. Jermaine, and Philip A. Bernstein (Eds.). ACM, 19–34. https://doi.org/10.1145/3183713.3196926
[47] Stephen H. Muggleton and Luc De Raedt. 1994. Inductive Logic Programming: Theory and Methods. *J. Log. Program.* 19/20 (1994), 629–679. https://doi.org/10.1016/0743-1066(94)90035-3
[48] Shan-Hwei Nienhuys-Cheng and Ronald de Wolf. 1997. *Foundations of Inductive Logic Programming*. Lecture Notes in Computer Science, Vol. 1228. Springer. https://doi.org/10.1007/3-540-62927-0
[49] Ralph Peeters, Aaron Steiner, and Christian Bizer. 2025. Entity Matching using Large Language Models. In *Proceedings 28th International Conference on Extending Database Technology, EDBT 2025, Barcelona, Spain, March 25-28, 2025*, Alkis Simitsis, Bettina Kemme, Anna Queralt, Oscar Romero, and Petar Jovanovic (Eds.). OpenProceedings.org, 529–541. https://doi.org/10.48786/EDBT.2025.42
[50] Eduardo H. M. Pena, Fábio Porto, and Felix Naumann. 2022. Fast Algorithms for Denial Constraint Discovery. *Proc. VLDB Endow.* 16, 4 (2022), 684–696. https://doi.org/10.14778/3574245.3574254
[51] Luc De Raedt. 2008. *Logical and relational learning*. Springer. https://doi.org/10.1007/978-3-540-68856-3
[52] Philipp Schirmer, Thorsten Papenbrock, Ioannis K. Koumarelas, and Felix Naumann. 2020. Efficient Discovery of Matching Dependencies. *ACM Trans. Database Syst.* 45, 3 (2020), 13:1–13:33. https://doi.org/10.1145/3392778
[53] Rohit Singh, Venkata Vamsikrishna Meduri, Ahmed Elmagarmid, Samuel Madden, Paolo Papotti, Jorge-Arnulfo Quiané-Ruiz, Armando Solar-Lezama, and Nan Tang. 2017. Synthesizing entity matching rules by examples. *Proc. VLDB Endow.* 11, 2 (Oct. 2017), 189–202. https://doi.org/10.14778/3149193.3149199
[54] Parag Singla and Pedro M. Domingos. 2006. Entity Resolution with Markov Logic. In *Proceedings of the 6th IEEE International Conference on Data Mining (ICDM 2006), 18-22 December 2006, Hong Kong, China*. IEEE Computer Society, 572–582. https://doi.org/10.1109/ICDM.2006.65
[55] Balder ten Cate, Maurice Funk, Jean Christoph Jung, and Carsten Lutz. 2023. Fitting Algorithms for Conjunctive Queries. *SIGMOD Rec.* 52, 4 (2023), 6–18. https://doi.org/10.1145/3641832.3641834
[56] Tommaso Teofili, Donatella Firmani, Nick Koudas, Paolo Merialdo, and Divesh Srivastava. 2026. Can we trust LLM Self-Explanations for Entity Resolution? arXiv:2606.01210 [cs.DB] https://arxiv.org/abs/2606.01210
[57] Zhiliang Xiang, Meghyn Bienvenu, Gianluca Cima, Víctor Gutiérrez-Basulto, and Yazmín Ibáñez-García. 2024. ASPEN: ASP-Based System for Collective Entity Resolution. In *Proceedings of the 21st International Conference on Principles of Knowledge Representation and Reasoning, KR 2024, Hanoi, Vietnam. November 2-8, 2024*, Pierre Marquis, Magdalena Ortiz, and Maurice Pagnucco (Eds.). https://doi.org/10.24963/KR.2024/74
[58] Zhiliang Xiang, Meghyn Bienvenu, Gianluca Cima, Víctor Gutiérrez-Basulto, and Yazmín Ibáñez-García. 2025. Advances in Logic-Based Entity Resolution: Enhancing ASPEN with Local Merges and Optimality Criteria. In *Proceedings of the 22nd International Conference on Principles of Knowledge Representation and Reasoning, KR 2025, Melbourne, Australia, November 1-17, 2025*, Magdalena Ortiz, Renata Wassermann, and Torsten Schaub (Eds.). https://doi.org/10.24963/KR.2025/64
[59] Dongxiang Zhang, Long Guo, Xiangnan He, Jie Shao, Sai Wu, and Heng Tao Shen. 2018. A Graph-Theoretic Fusion Framework for Unsupervised Entity Resolution. In *34th IEEE International Conference on Data Engineering, ICDE 2018, Paris, France, April 16-19, 2018*. IEEE Computer Society, 713–724. https://doi.org/10.1109/ICDE.2018.00070

A COMPUTATION OF CORE (CF SECTION 5)

It is useful to introduce the notion of *pairing*, which captures when two ER rules have matching structures, i.e., when the variables of one rule can be mapped to those of the other.

Definition 1. *Let $\Gamma(\mathcal{S}, \tau)$ be a target space. For any two ER rules $r_1, r_2 \in \Gamma(\mathcal{S}, \tau)$, a* pairing *is a pair of atoms $(a, a') \in body(r_1) \times body(r_2)$ such that (a, a') is called: (i) a relational pairing if a and a' are relation atoms over the same relation symbol; or (ii) a comparison pairing if a and a' are comparison atoms over the same attributes with the same comparison operator.*

Algorithm 2 AllPairings

Input: Two ER rules r_1, r_2 over a target space $\Gamma(\mathcal{S}, \tau)$
Output: The pair of all pairings of r_1 and r_2
$A := \{(a, a') | (a, a') \in rel(r_1) \times rel(r_2)$ and a, a' have the same relation$\}$
$C := \emptyset$
for each $(a, a') \in A$ **do**
  $C_1 := \{c | c \in comp(r_1), var(a) \subset var(c)\}$
  $C_2 := \{c' | c' \in comp(r_2), var(a') \subset var(c')\}$
  for each $(c, c') \in C_1 \times C_2$ **do**
    if c, c' have the same comparison on the same attributes **then**
      $C := C \cup \{(c, c')\}$
    end if
  end for
end for
return (A, C)

Algorithm 3 ComputeCore

Input: An ER rule r
Output: A core $r^\star$ of r
$r' := r$, $h := \emptyset$
repeat
  $A, C :=$ AllPairings(r', r')
  for each $(a, a') \in A$ and $var(a') \neq var(a)$ **do**
    $h' := h \cup \{var(a') \to var(a)\}$
    if $r' \succeq rh'$ **then**
      $h := h'$, $r' := rh$
      break
    end if
  end for
until no further change on h
for each comparison atom $c \in body(r')$ **do**
  $r'' := head(r') \leftarrow body(r') \setminus \{c\}$
  if $r' \succeq r''$ w.r.t. identity mapping **then**
    $r' := r''$
  end if
end for
return r'

We represent a set of pairings between two ER rules r_1 and r_2 from the same target space by a pair $S = (A, C)$, where A and C are the sets of relational and comparison pairings, respectively. We say that S is *valid* if both A and C are non-empty, and every variable occurring in a comparison pairing in C also occurs in a relational pairing in A. The (pair of) all pairings of r_1 and r_2, is a pair (A^*, C^*) s.t. A^* and C^* are $\subseteq$-maximal set of relational and comparison pairings in r_1 and r_2, respectively. Denote $rel(r)$ and $comp(r)$ the set of relational atoms and comparison atoms in $body(r)$, respectively. Algorithm 2 outlines a procedure for computing all pairings between two ER rules from the same target space. With these notions, we describe ComputeCore(r), which returns a core in the $\succeq$ induced equivalence class of an input ER rule (cf Section 5). In Algorithm 3, the first for-loop (Lines 6–12) repeatedly unifies tuple variables in $body(r)$ until no further unification is possible. Each iteration

produces a rule r' that is $\succeq$-equivalent to r, while the corresponding variable mapping is accumulated in h. Consequently, for any mapping h' that unifies at least two distinct variables in $body(r)$, we have $h' \subseteq h$ for the resulting mapping h. To eliminate redundant comparison atoms, the second for-loop (Lines 14–18) iterates over each comparison atom $c \in body(r')$ and checks whether removing c yields a rule r'' that remains $\succeq$-equivalent to r'.

Lemma 4. *Given an ER rule r in a target space $\Gamma(\mathcal{S}, \tau)$, ComputeCore$(r)$ computes the core of r in its equivalence class under $\langle \Gamma(\mathcal{S}, \tau)/_{=}, \succeq \rangle$.*

Proof. Let $r' =$ ComputeCore(r). Suppose, for contradiction, that there exists an ER rule r'' such that r' and r'' are $\succeq$-equivalent and $|body(r'')| < |body(r')|$. Then there exists a mapping $h : var(r') \rightarrow var(r'')$ such that $r'h$ satisfies the definition of $\succeq$. Since $|body(r'')| < |body(r')|$, we also have $|body(r'h)| < |body(r')|$. Moreover, $r'h$ is $\succeq$-equivalent to r', and there exists a mapping $h' : var(r'') \rightarrow var(r')$ such that $r'h \succeq r'$. For such a mapping h' to satisfy the definition of $\succeq$, the composed mapping hh' must produce a rule $r'hh'$ in which either (i) at least two distinct variables of r' are unified, or (ii) some comparison atom $c \in body(r')$ is redundant, i.e., c can be removed while preserving $\succeq$-equivalence. However, neither can occur for the rule r' produced by Algorithm 3. □

B PROOF

Proof for Section 3

Proposition 1. *Let $\mathcal{S}$ be a schema, D an $\mathcal{S}$-database, and Σ a set of ER rules over $\mathcal{S}$. Then (c, c') is in the static solution for (D, Σ) iff $\mathsf{eqo}(\mathsf{c}, \mathsf{c}') \in M$ for some answer set M of $\Pi_s(D, \Sigma)$.*

Proof. Let D be an $\mathcal{S}$-database, Σ a set of ER rules, and $\Pi_s(D, \Sigma)$ the ASP encoding defined in Section 3.1 (which always has a unique answer set). The encoding represents every database tuple and similarity fact included in D as ASP facts, and derives an atom $\mathsf{eqo}(\mathsf{c}, \mathsf{c}')$ exactly when (c, c') in E_s the static solution for (D, Σ). Hence, there is a one-to-one correspondence between the pairs in E_s and the atoms $\mathsf{eqo}(\mathsf{c}, \mathsf{c}')$ appearing in an answer set of $\Pi_s(D, \Sigma)$. □

Proposition 2. *Let $\mathcal{S}$ be a schema, D be an $\mathcal{S}$-database and (Σ, Δ) be a pair of ER rule set and DCs over $\mathcal{S}$. Then (i) (c, c') is in the maximal solution for (D, Σ) iff $\mathsf{eqo}(\mathsf{c}, \mathsf{c}') \in M$ for some answer set M of $\Pi_d(D, \Sigma)$. (ii) A consistent solution $E \in \mathsf{ConSol}(D, \Sigma, \Delta)$ iff $E = \{(c, c') | \mathsf{eqo}(\mathsf{c}, \mathsf{c}') \in M\}$ where M is an answer set of the ASP encoding $\Pi_d(D, (\Sigma, \Delta))$.*

Proof. For (i), given E the maximal solution for (D, Σ), a sequence of building E $E_0, e_0, ..., e_n, E_n$ s.t. $E_0 = \mathsf{EqRel}(\emptyset, D)$ and for every $i \in [1, n]$, $E_{i+1} = \mathsf{EqRel}(E_i \cup \{e_i\}, D)$ for some $e_i \in r(D_{E_i})$. By suitably enumerating the pairs in E_0 and each $E_{i+1} \setminus E_i \cup e_i$, we obtain an enumeration of eqo-fact in the answer set of $\Pi_d(D, \Sigma)$:

$$\mathsf{eqo}(\alpha_1^{\emptyset}), ..., \mathsf{eqo}(\alpha_{\mathsf{p}_0}^{\emptyset}), ..., \mathsf{eqo}(\mathsf{e}_1), ...,$$
$$\mathsf{eqo}(\alpha_1^{1}), ..., \mathsf{eqo}(\alpha_{\mathsf{p}_1}^{1}), \mathsf{eqo}(\mathsf{e}_2), ...,$$
$$\mathsf{eqo}(\mathsf{e}_n), ..., \mathsf{eqo}(\alpha_1^{n}), ..., \mathsf{eqo}(\alpha_{\mathsf{p}_n}^{n})$$

such that each fact can be derived from D and the preceding facts in the enumeration using the ground rules in the reduct of $gr(\Pi_d(D, \Sigma))$. Each $\alpha_{\mathsf{k}}^{\emptyset}$ is a reflexive pair, and each $\mathsf{eqo}(\alpha_{\mathsf{k}}^{\mathsf{j}})$ with $j > 0$ is obtained via grounding of the transitive rule or the symmetry rule (cf Section 3.1), each $\mathsf{eqo}(\mathsf{e_i})$ with an e_i added to E_i due to an ER rule in Σ by the instantiation of its ASP encoding. The converse direction follows analogously. The proof of (ii) is analogous. □

Proof for Section 5

Lemma 1. *Let $\mathcal{S}$ be a schema and $\Gamma(\mathcal{S}, \tau)$ be the target space identified by τ. For any non-empty finite subset $\gamma \subseteq \Gamma(\mathcal{S}, \tau)$, there exists a rule $r_l \in \Gamma(\mathcal{S}, \tau)$ such that r_l is a minimal GS of γ.*

Proof. Let $\gamma = \{r_1, ..., r_n\}$ be non-empty finite subset of $\Gamma\langle \mathcal{S}, \tau \rangle$. Assume wlog that variable occurrences among γ are distinct. Let g be a mapping that unifies every pair of head variables, say to x and y, for each $r_i \in \gamma$ where $i \in [1, n]$. For each r_i, we apply g and obtain $r_i g$. Let r_{ub} be $x[A] = y[B] \leftarrow body(r_1 g) \cup ... \cup body(r_n g)$. It is easy to see that $r_i \succeq r_{ub}$ for each $r_i \in \gamma$ as g only acts on head variables of r_i, and $body(r_i g)$ and $body(r_i)$ are $\succeq$-equivalent.

Suppose for some r'_{ub} we have $r_i \succeq r'_{ub}$ for every $r_i \in \gamma$. There must exist for each r_i a mapping h_i from $var(r_i)$ to $var(r'_{ub})$ such that the rule rh_i obtained by applying the mapping satisfies the definition of $\succeq$. Let $h = h_1 \cup ... \cup h_n$, and let $head(r'_{ub})$ be $x'[A] = y'[B]$. Then rh is the rule $x'[A] = y'[B] \leftarrow body(r_1 h) \cup ... \cup body(r_n h)$, which subsumes r'_{ub}. Now, let m be a mapping matches only x, y from r_{ub} to x', y' to r'_{ub}. The rule $r_{ub}m$ satisfies the definition of $\succeq$, since for every $body(r_i g m)$ remains $\succeq$-equivalent to $body(r_i)$. Hence, $r_{ub} \succeq r'_{ub}$. □

Lemma 2. *Let $\mathcal{S}$ be a schema and $\Gamma(\mathcal{S}, \tau)$ be the target space identified by τ. For any non-empty finite subset $\gamma \subseteq \Gamma(\mathcal{S}, \tau)$, there exists rule $r_u \in \Gamma(\mathcal{S}, \tau)$ such that r_u is a minimal LG of γ.*

Proof. Let $r'_1 = \mathsf{cl}(r_1), r'_2 = \mathsf{cl}(r_2)$. Clearly, r'_1 and r_1 (r'_2 and r_2 resp.) are $\succeq$-equivalent. Hence, it suffices to exhibit a LG for $\{r'_1, r'_2\}$.

Let $S = (A, C)$ a valid pair of pairings of any two rules $r, r' \in \Gamma(\mathcal{S}, \tau)$. From S we construct rules:

- $r_s = head(r) \leftarrow a_1, ..., a_n, c_1, ..., c_m$,
- $r'_s = head(r') \leftarrow a'_1, ..., a'_n, c'_1, ..., c'_m$

, where each $a_i, a'_i \in A$ and each $c_j, c'_j \in C$, and $a_i, c_j \in body(r)$, $a'_i, c'_j \in body(r')$. By construction, r_s, r'_s are $\succeq$-equivalent.

Claim 1: *For any valid pair of pairings S of r_1, r_2, let r_s be a rule constructed from S as above, then $r_s \succeq r_1$ and $r_s \succeq r_2$.*

Since $body(r_s)$ contains those atoms from either r_1 or r_2, hence either $r_s \succeq r_1$ or $r_s \succeq r_2$. Because in either case r_s is $\succeq$-equivalent to a rule contains only atoms from the other, say $r_s = r_{s1}$, and r_{s1} and r_{s2} are $\succeq$-equivalent, we also have that $r_s \succeq r_2$.

Claim 2: *Let $S_1 = (A_1, C_1), S_2 = (A_2, C_2)$ be any two valid pairs of pairings of r_1, r_2, if $A_1 \subseteq A_2$ and $C_1 \subseteq C_2$, then $r_{s1} \succeq r_{s2}$ for the rules r_{s1}, r_{s2} constructed from S_1, S_2, respectively.*

Since $A_1 \subseteq A_2$ and $C_1 \subseteq C_2$, there exists a mapping s.t. $body(r_{s1}) \succeq body(r_{s2})$ and for such mapping h if $head(r_{s1})h = head(r_{s2})h$, then by the definition of $\succeq$, we also have that $r_{s1} \succeq r_{s2}$.

Claim 3: *For every rule r s.t. $r \succeq r_1$ and $r \succeq r_2$, there exists a valid pair S of pairings of r'_1 and r'_2 s.t. r_s and r are $\succeq$-equivalent for a rule r_s constructed from S.*

Since $r \succeq r'_1$, we have that there exists a mapping h_1, s.t. for each atom $a_i h_1$ where $a_i \in body(r)$, and $|body(r)| = n$, there exists $a'_i \in body(r'_1)$. Analogously, there exists a mapping h_2, s.t. for each

atom $a_i h_2$ where $a_i \in body(r)$, there exists $a_i'' \in body(r_2')$. Let S be the pair of pairings of r_1' and r_2' $((a_1', a_1''), ..., (a_n', a_n''))$, and r_s the rule constructed from S. r_s and r are $\succeq$-equivalent since for every body atom a_i' of r_s can be mapped to $a_i \in body(r)$, and vice versa.

Denote $rel(r)$ and $comp(r)$ the set of relation atoms and comparison atoms in $body(r)$ for a given rule r, respectively. Following Claim 2 and Claim 3, we have the Claim 4 below holds:

Claim 4: *Let S^* the pair of all pairings of r_1' and r_2', which can be computed by Algorithm 2, then a rule constructed from S^* is a LG of $\{r_1, r_2\}$.* □

Lemma 3 *Let $r \in \Gamma(\mathcal{S}, \tau)$. There exists a rule $r' \in \Gamma(\mathcal{S}, \tau)$ and a mapping from $var(r)$ to $var(r')$ such that r and r' are $\succeq$-equivalent and r' is a core.*

Proof. The claim follows immediately from Lemma 4, which shows that ComputeCore(r) returns a core of r. □

Theorem 2. *Let $\mathcal{S}$ be a schema and τ a target over $\mathcal{S}$. For every $r, r' \in \Gamma(\mathcal{S}, \tau)$ and every $\mathcal{S}$-database D, it holds that $r \succeq r'$ iff $r'(D) \subseteq r(D)$.*

Proof. Let D be an $\mathcal{S}$-database. And assume $\tau = (R, A, R', A')$. We can assume that

$r = x[A] = y[A'] \leftarrow \varphi(x, y)$
$r' = x[A] = y[A'] \leftarrow \varphi'(x, y)$
$\text{cl}(r') = x[A] = y[A'] \leftarrow \varphi''(x, y)$

(⇒) We start by proving that $r \succeq r'$ implies $r'(D) \subseteq r(D)$.

It suffices to show that for every pair of constant tuples $\vec{c_1} \in R^D$, and $\vec{c_2} \in R'^D$, if $\varphi'(\vec{c_1}, \vec{c_2})$ is satisfied in D then $\varphi(\vec{c_1}, \vec{c_2})$ is also satisfied in D.

Towards a contradiction, let $\vec{c_1} \in R^D$, and $\vec{c_2} \in R'^D$ be tuples such that if $\varphi'(\vec{c_1}, \vec{c_2})$ is satisfied in D but $\varphi(\vec{c_1}, \vec{c_2})$ is not satisfied in D.

We observe that by the definition of closure and semantics of comparison atoms, it holds that $\varphi''(\vec{c_1}, \vec{c_2})$ is satisfied in D.

Now, let $h : var(r) \to var(r')$ be a mapping witnessing $r \succeq r'$. We have that $h(\varphi) \subseteq \varphi''$. But since h only renames variables in φ, we would have that $\varphi''(\vec{c_1}, \vec{c_2})$ is not satisfied in D.

(⇐) Next we show that If $r'(D) \subseteq r(D)$, then $r \succeq r'$.

Given an input $\bar{r}$, consider the following to obtain a rule $\bar{r}_e$ that is $\succeq$-equivalent to $\bar{r}$:

(1) Let $\beta = \epsilon(body(\bar{r}))$.
(2) For every join atom of the form $x[A] = y[B] \in \beta$, extend β by adding the similarity atom $\beta = \beta \cup \{x[A] \approx y[B]\}$.
(3) Let $\bar{r}_e = head(\bar{r}) \leftarrow \beta$.

Given two ER rules over $\mathcal{S}$, let r_e, r_e' the two rules constructed as above. It suffices to show that Claim 2 holds for r_e, r_e'. Assume $r_e'(D) \subseteq r_e(D)$ for every $\mathcal{S}$-database D and $r_e \not\succeq r_e'$. Since $r_e \not\succeq r_e'$, there must be either a relation atom or a comparison atom in $body(r_e)$ that cannot make a pairing with any atoms in $body(r_e')$. Now, consider an $\mathcal{S}$-database $D_{r_e'}$ constructed as following: For each $R(x) \in r_e'$, let $R(t) \in D_{r_e'}$ where in tuple t

- each attribute A of tuple t_x is assigned to a constant sharing across the equivalence class of $x[A]$ in r_e', that is, $t_x[A] := c_{x[A], x_1[A_1], ..., x_n[A_n]}$, for each $x_i[A_i]$ s.t. $x[A] = x_i[A_i] \in body(r_e')$ and $i \in [1, n]$
- For every other attribute A' of tuple t_x that does not participate in any join atom in r_e', $t[A']$ is assigned to a constant uniquely identified by the term $t[A'] := c_{x[A']}$.

Denote $free(r)$ the set of free variables occur in an ER rule r. Assume, wlog, for every similarity atom $x[A] \approx y[B] \in body(r_e')$, $t_x[A] \approx t_y[B]$ holds where $t_x, t_y \in D$. Since $r_e'(D) \subseteq r_e(D)$ for every D, we have that $r_e'(D_{r_e'}) \subseteq r_e(D_{r_e'})$. Therefore, there exists mappings $v : var(r) \to D_{r_e'}$, such that $v(free(r_e)) \in D_{r_e'}$. But this contradicts to the assumption that there exists atoms in r_e cannot make pairings with any atoms in r_e'. The database $D_{r_e'}$ contains only tuples generated from atoms of r_e', therefore no mapping v can match the atoms in r_e with no pairings in r_e', contradicting the existence of such a mapping. □

Proposition 3. *Let $\mathcal{S}$ be a schema and τ a target over $\mathcal{S}$. For every $r, r' \in \Gamma(\mathcal{S}, \tau)$, and every $\mathcal{S}$-database. If $r \succeq r'$, then $r'(D_E) \subseteq r(D_E)$ for any equivalence relation E over $\mathsf{adom}(D)$.*

Proof. For contradiction, assume there exist an equivalence relation E, an $\mathcal{S}$-database D, and a pair $a = (c_x, c_y)$ such that $a \in r'(D_E)$ but $a \notin r(D_E)$Let $\bar{a} = (\bar{\epsilon}(c_x), \bar{\epsilon}(c_y))$. We have that $\bar{a} \in r'(D_E)$ but $\bar{a} \notin r(D_E)$, contradicting to Theorem 2. Therefore, no such a exists and we have $r'(D_E) \subseteq r(D_E)$ as required. □

Proof for Section 6

Proposition 4. *Given a learning instance $\langle \mathcal{S}, \tau, D, \mathsf{Ex}^+, \mathsf{Ex}^- \rangle$, every answer set of $\Pi_{\mathsf{meta}}(\mathcal{S}, \tau)$ (via its meta atoms) corresponds to an ER rule in $\Gamma(\mathcal{S}, \tau)$. Moreover, for a given number n of distinct variable names, every $r \in \Gamma(\mathcal{S}, \tau)$ with $size(r) \leq \frac{n+2}{2}$ is represented by an answer set of $\Pi_{\mathsf{meta}}(\mathcal{S}, \tau)$.*

Proof. **Soundness.** Let $\Pi_{\mathsf{meta}}(\mathcal{S}, \tau)$ be the rule generation program for $\Gamma(\mathcal{S}, \tau)$. Assume that the schema declarations in $\Pi_{\mathsf{meta}}(\mathcal{S}, \tau)$ are complete as described in Section 6.1, and that $\mathtt{num\_vars}(n) \in \Pi_{\mathsf{meta}}(\mathcal{S}, \tau)$ for some $n > 4$. Without loss of generality, let $\tau = (R, A, R', A')$.

For every answer set $M \in SM(\Pi_{\mathsf{meta}}(\mathcal{S}, \tau))$, the generation rules derive the meta atom $\mathtt{head}(\mathtt{eqo}, 2, (\mathtt{0}, 1))$. together with the head-relevant body atoms

$$\mathtt{body}(\mathsf{r}, 1, 2).\ \mathtt{body}(\mathtt{att}, \mathsf{a}, (2, \mathsf{a}, \mathtt{0})).$$
$$\mathtt{body}(\mathsf{r}', 1, 3).\ \mathtt{body}(\mathtt{att}, \mathsf{a}', (3, \mathsf{a}', 1)).$$

which encode ER rules of the form $x[A] = y[A'] \leftarrow R(x), R'(y)$. The remaining generation rules (Lines 9–16) in Figure 2 enumerate all possible expansions of this rule that are representable using the specified number of variable names, thereby over-approximating $\Gamma(\mathcal{S}, \tau)$.

The constraints in Lines 1–10 of Figure 7 eliminate invalid expansions by requiring that join and similarity atoms are defined only on compatible attributes (Lines 1–4), and that the comparison atoms induce a connected connectivity graph (Lines 5–10). Hence, every answer set represents a connected ER rule over the target τ.

Finally, Lines 14–17 impose an ordering on the variable names by requiring them to be introduced incrementally. Together with the constraints in Lines 27–29, every newly added tuple variable (`tail`) must lie on distinct non-diverging paths to the head variables. Now,

every $M \in SM(\Pi_{\text{meta}}(\mathcal{S},\tau))$ represents an ER rule r over the target τ, and r is biconnected, satisfying biconnectivity.

Bounded completeness. Lines 9–16 of Figure 2 enumerate all possible expansions of the base rule $r_b : x[A] = y[A'] \leftarrow R(x), R'(y)$, that are representable using n variable names. The constraints in Figure 7 eliminate only those answer sets encoding syntactically invalid or non-biconnected ER rules. Therefore, every biconnected ER rule extending r_b that is representable using n variable names is represented by an answer set of $\Pi_{\text{meta}}(\mathcal{S},\tau)$. To derive a sufficient bound on the rule size, observe that the base rule r_b requires two variable names. Each additional similarity atom increases the rule size by one and, in the worst case, introduces two new variable names, namely when the similarity atom relates a previously unseen pair of attributes. Thus, an ER rule of size m requires at most $2 + 2(m-2)$ variable names. Consequently, every ER rule with $m \leq \frac{n+2}{2}$ is representable using n variable names. Hence, $\Pi_{\text{meta}}(\mathcal{S},\tau)$ is complete for all such ER rules. □

Proposition 5. *Let $\mathcal{S}$ be a schema, $\Gamma(\mathcal{S},\tau)$ the target space defined by $(\mathcal{S},\tau)$, and $\Pi_{\text{meta}}(\mathcal{S},\tau)$ the corresponding generation program. For every $M_r \in SM(\Pi_{\text{meta}}(\mathcal{S},\tau))$ representing a rule $r \in \Gamma(\mathcal{S},\tau)$, a generalisation (resp. specialisation) constraint constructed from M_r prunes generalisations (resp. specialisations) of r, and never prunes any rule $r' \in \Gamma(\mathcal{S},\tau)$ such that $r \succ r'$ (resp. $r' \succ r$).*

Proof. **GC soundness.** Let $M_r \in SM(\Pi_{\text{meta}}(\mathcal{S},\tau))$ represent an ER rule $r \in \Gamma(\mathcal{S},\tau)$, and let $\gamma_w \subset \Gamma(\mathcal{S},\tau)$ denote the set of ER rules obtained from r by weakening one or more join atoms into similarity atoms while preserving $size(r)$. Observe the GC encoding in Section 6.2 eliminates two kinds of answer sets. (i) Answer sets representing ER rules r' such that $r' \equiv r$. Every such r' is a generalisation of r, and $r' \not\succ r$. (ii) Answer sets representing ER rules r' such that, whenever $\gamma_w \neq \emptyset$, there exists $r_w \in \gamma_w$ with $r_w \equiv r'$. By construction, a mapping $h : var(r') \rightarrow var(\text{cl}(r))$ can be obtained by enumerating the combinations produced by Algorithm 2, yielding $body(r'h) \subseteq body(\text{cl}(r))$. The converse does not hold because the join atoms in $body(r)$ have no corresponding atoms in $body(\text{cl}(r'))$. Hence, $r' \succeq r$, but $r \not\succ r'$. Therefore, every rule pruned by a GC is a generalisation of r, while no specialisation of r is pruned.

SC soundness. Let $M_r \in SM(\Pi_{\text{meta}}(\mathcal{S},\tau))$ represent an ER rule $r \in \Gamma(\mathcal{S},\tau)$, and let r^* denote the core of r (hence $r \equiv r^*$). Let $\gamma_s \subseteq \Gamma(\mathcal{S},\tau)$ denote the set of ER rules obtained from r^* by strengthening one or more similarity atoms into join atoms. By construction, $r^* \succ r_s$ for every $r_s \in \gamma_s$, whenever $\gamma_s \neq \emptyset$. Similar to the GC encoding, the SC encoding in Section 6.2 eliminates two kinds of answer sets. (i) Answer sets representing ER rules r' such that $r^* \succeq r'$. Every such r' is a specialisation of r, and $r' \not\succ r$. (ii) Answer sets representing ER rules r' such that, whenever $\gamma_s \neq \emptyset$, there exists $r_s \in \gamma_s$ with $r_s \succeq r'$. Since $r^* \succ r_s$ for every $r_s \in \gamma_s$, it follows that $r^* \succ r'$, and hence every such r' is a specialisation of r. Therefore, $r' \not\succ r$. Therefore, every rule pruned by an SC is a specialisation of r, while no generalisation of r is pruned. □

Theorem 3. *Given a learning instance $\mathcal{I}$, a maximal rule size m, and approximation parameters (θ,δ), Algorithm LeCER returns an optimal (approximation) ruleset in $\mathcal{I}$ if one exists.*

Proof. Since LeCER performs a level-wise search, every candidate ER rule of size at most m is eventually generated unless pruned by a sound generalisation or specialisation constraint (Proposition 5). Whenever a generated ER rule covers all positive examples and is (θ,δ)-consistent (Line 18), the algorithm immediately returns the singleton ruleset $\{r\}$. Since rules are generated in increasing order of size, no smaller rule satisfying these conditions exists, and $\{r\}$ is therefore an optimal ruleset. Otherwise, every supported and (θ,δ)-consistent rule is added to the candidate set Σ. Let E_s denote the static solution of (D,Σ). The Combine function computes a subset $\Sigma^* \subseteq \Sigma$ whose static solution E_s^* satisfies $E_s \subseteq E_s^*$, as enforced by the hard clauses, while optimising the objective. If E_s already contains all positive examples, the soft clauses for example coverage are inactive, and Combine minimises only the size of the ruleset. Otherwise, it maximises the coverage of positive examples while minimising the ruleset size. Hence, Combine returns an optimal (approximation) ruleset whenever one exists. □

C ASP PROGRAM FOR ER SYNTACTIC CONSTRAINTS IN SECTION 6.1

Syntactic constraints. The ASP rules in Figure 2 compute an

```
:- body(att,3,(T,A,V)), body(att,3,(T',A',V)), not comp_a(A,A').
:- body(att,3,(T,A,V)), body(att,3,(T',A',V')), body(sim,2,(V,V')),
      not comp_a(A,A').
:- body(R,1,T), not body(att,3,(T,_,_)).

% connectivity
adj(T,T') :- body(att,_,(T,_,V)), body(att,_,(T',_,V)).
adj(T,T') :- body(att,_,(T,_,V)), body(att,_,(T',_,V')), body(sim,_,(V,V')).
connected(T,T') :- adj(T,T').
connected(T,T') :- adj(T,T'), connected(T',T).
:- not connected(T,T'), body(R,1,T), body(R',1,T'), T!=T'.
pk
% bi-connectivity
% enforce order over variables used in a rule
used_var(V) :- head_var(V).
used_var(V) :- body_var(Var).
:- used_var(V), V > 1, V' = 1..V-1, not used_var(V').

tail(T) :- T = #max{T': body(R,1,T')}, body(R,1,T).
head_tv(T) :- body(R,1,T), target(R,A,V), body(att,3,(T,A,V)).
pairwise :- head_tv(T), tail(T).
edge(T,T') :- adj(T,T'), T>T', not pairwise.

{ path(T,T',TS) : edge(T,T') }=1 :- not pairwise, head_tv(TS), tail(T).
{ path(T,T',TS) : edge(T,T') }=1 :- path(_,T,TS), T <> TS.

:- path(X,Y,T), path(X,Z,T), Y <> Z.
:- path(X,Y,T), path(Z,Y,T), X <> Z.
:- path(X,Y,T), path(X',Y,T'), T<>T'.
```

Figure 7: ASP constraints for syntactic constraints of ER rules

over-approximation of the target space consisting ER rules with unrestricted structures like those are not connected, while ASP constraints shown in Figure 7 refine the space. In Figure 7, Lines 1-4 eliminate ER rules that involve comparisons on any attributes that are not compatible, and connectivity are enforced by Lines 6-11. In particular, Lines 7-8 define adjacency relations between tuple variables based on comparison atoms, then connectivity is recursively propagated in Line 10. Finally, Line 11 eliminates ER rules whose tuple variables are not all connected. The restriction of bi-connectivity

requires a more elaborate encoding (Lines 13-29). First, Lines 14–17 impose an ordering over variables used in an ER rule, by enforcing that variable names are introduced incrementally. Hence, whenever a fresh variable is added to an ER rule, its variable name must be exactly one greater than the largest variable name already used. Then, the tuple variable with the largest variable name is selected as the tail (Line 19). If the tail itself is a tuple variable from the head, the ER rule only includes a pair of tuple variables, therefore no constraint will be imposed (Line 21). Otherwise, a directed graph is built from the undirected connectivity graph by orienting edges from larger to smaller tuple variable names (Line 22). Paths are then generated from the tail towards head tuple variables (Lines 24–25). The remaining constraints ensure that the resulting structure forms a valid non-diverging path: Lines 27-29 prevent path branching by enforcing functional predecessors and successors, while Line 29 guarantees that two distinct paths to different head tuple variables cannot merge at an intermediate node, thereby constrain added tuple variables to be adjacent with nodes belong to both "components" attached to the two head variables.